Oracle PL SQL

Chapter 1: Introduction (4-10)
What is PL/SQL? 4
PL/SQL Basic Syntax (Block) 5
Nested PL/SQL Block 8
Advantages Of PL/SQL 10

Chapter 2: PL/SQL Variables & Data Types (11-15)
What are Variables? 11
Scalar Data Types 12
Reference Data Types 13
LOB Data Types 14
Constant Variables 14
Scope Of Variables 15

Chapter 3: Conditional Statements (17-23)
Conditional Statements 17
ONLY IF STATEMENT 17
IF – ELSE 19
NESTED IF 20
ELSIF 21
CASE Statement Based On Variable 22
CASE Statement Based on Condition 23

Chapter 4: Iterative Statements (25-26)
Iteration Control Statements (LOOP) 25
Simple Loop 25
While Loop 26
For Loop 26

Chapter 5: GoTo Statements (27-28)
GOTO And LABELS 27
Restrictions for using Goto 28

Chapter 6: Cursors (29-37)
Cursors 29
Implicit Cursor 30
Explicit Cursor 32
Cursor using FOR Loop 35
Parameterized Cursors.....37

Chapter 7: Error Handling (39-44)
What is an EXCEPTION? 39
Pre-Defined Exceptions 40
Unnamed system exceptions 43
User Defined Exceptions 44

Chapter 8: Record Types (45-47)
Record Type In PL/SQL.....45
Record Using %ROWTYPE 47
Record Using General Syntax 47

Chapter 9: Sub-Programs (48-55)
Stored Procedure 48
PL/SQL Function 52
Parameters In Procedures And Functions 54
Subprogram Dependences 55

Chapter 10: Packages (56-61)
What is a Package 56
Components Of A Package 57
Package Syntax 58

Package Example 58
Scope of various objects in a package 60
Advantages Of Packages 61

Chapter 11: Collections (62-76)
What is a collection? 62
Collections Methods 63
VARRAYS 70
Nested Tables 73
Index By Tables or Associate Arrays 74
Differences Among collections 76

Chapter 12: Triggers (76-81)
What is a Trigger 76
ROW LEVEL TRIGGER 78
STATEMENT LEVEL TRIGGER 79
Trigger Execution Hierarchy 80
Mutating Triggers 81

Chapter 1: Introduction

What is PL/SQL?

Before we discuss about PL/SQL, Lets first see what is that

- SQL cannot do
- And the need for PL/SQL

Requirement : 'Update Salaries of all employees by 2000'

- To achieve this requirement we can write a simple UPDATE statement in SQL.

```
UPDATE emp
SET sal = sal+2000;
```

New Requirement : 'Update Salaries of employees based on the designation as below'

DESIGNATION	INCREMENT
MANAGER	10000
ANALYST	5000
CLERK	2000
SALESMAN	No increment

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7369	SMITH	CLERK	7902	2/20/1981	2400		20
7499	ALLEN	SALESMAN	7698	2/20/1981	2160	300	30
7521	WARD	SALESMAN	7698	2/22/1981	1620	500	30
7566	JONES	MANAGER	7839	4/2/1981	3075		20
7654	MARTIN	SALESMAN	7698	9/28/1981	1620	1400	30
7698	BLAKE	MANAGER	7839	5/1/1981	2950		30
7782	CLARK	MANAGER	7839	6/9/1981	2550		10
7788	SCOTT	ANALYST	7566	12/9/1982	3100		20
7839	KING	PRESIDENT					10
7844	TURNER	SALESMAN					[illegible]
7876	ADAMS	CLERK	[illegible]				20
7900	JAMES	CLERK	7698	[illegible]981	[illegible]		30
7902	FORD	ANALYST	7566	12/3/1981	3100		20
7934	MILLER	CLERK	7782	1/23/1982	1680		10

Wanted to increment Salary based on JOB.

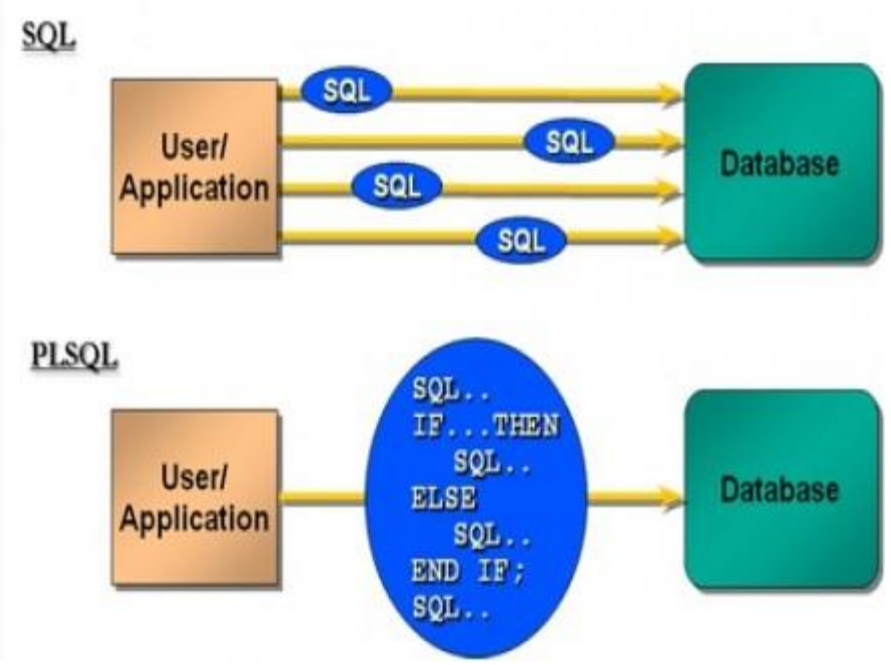
Question raises, can we achive this new requirement using a simple UPDATE statement ?

- No we cannot achieve this using a simple update statement.
- One alternative that we can think of is running multiple update statements (A separate update statement for each designation).

```
UPDATE emp
SET sal := sal+10000
```

WHERE job = 'MANAGER';

Finally, Answer to our 'New Requirement' is that we can write a pl/sql program which can update salaries of different designation employees at once using program constructs – 'IF ELSE' and 'LOOP'.



When a SQL statement is issued on the client computer, the request is made to the database on the server and the result set is sent back to the client.

As a result, a single SQL statement causes two trips on the network. If multiple SELECT statements are issued, network traffic increases significantly very fast. For example, four SELECT statements cause eight network trips.

If these statements are part of the PL/SQL block, they are sent to the server as a single unit. The SQL statements in this PL/SQL program are executed at the server and the result set is sent back as a single unit. There is still only one network trip made as is in case of a single SELECT statement.

PL/SQL Tutorial - PL/SQL Basic Syntax (Block)

In the previous lesson we discussed about 'the need of PL/SQL', In this lesson we will be learning PL/SQL basics.

Oracle PL/SQL is a wrapper to SQL, that is adding program constructs to SQL. Now for every program that we write in PL/SQL has a basic unit called as 'BLOCK'.

A set of SQL and PL/SQL statements are grouped together as a unit called BLOCK to solve a specific problem or to perform set of actions.

A PL/SQL Block consists of three sections:

- The Declaration section (Optional).
- The Execution section (Mandatory).
- The Exception (or Error) Handling section (Optional).

Declaration Section:

- The PL/SQL Block Declare section should start with keyword called 'DECLARE'.
- Declaration section is not mandatory in a block.
- Declaration section is used to declare any placeholders (that stores data temporarily) like variables, constants, records and cursors which are used in the execution section.

Execution Section:

- The PL/SQL Block Execution section should start with keyword 'BEGIN' and should end with keyword 'END'.
- This section is a mandatory and it is used to write the programming logic to achieve business requirement.
- Constructs like loops, conditional statement and SQL statements are part of execution section.

Exception Section:

- The PL/SQL Block Exception section should start with keyword 'EXCEPTION'.
- Exception section is not mandatory. This section is used to handle any error/exception raised in the program.
- Exception handling session is the place where exception handlers are defined to handle the run time errors.

E.g. like displaying an error message or sending custom mail notification to admin describing the error/exception.

- If there is no exception section defined to handle errors raised in PL/SQL Block then Block terminates abruptly. We will discuss in brief about exceptions in the coming chapters.
- There is no input statement for the PL/SQL, then to test the PL/SQL we use the plus command '&' is used.
- There is no print statement in PL/SQL but there is a debugging statement in the PL/SQL.

- The debugging statement of PL/SQL IS
  DBMS_OUTPUT.PUT_LINE();
- The debugging statement require a single parameter like VARCHAR2 (or) NUMBER (or) DATE.
- The debugging statement is a packaged procedure

  DBMS_OUTPUT : Package

  PUT_LINE : Procedure

- Note that we are going to learn about Packages and Procedures in the coming chapters.
- If you are using SQL*Plus then you must set the spool command " SET SERVEROUPUT ON" to see the debugging statement output.

A simple PL/SQL block example with out DECLARATION AND EXCEPTION section (as these are not mandatory) :

```
BEGIN

   DBMS_OUTPUT.PUT_LINE('Welcome to - Provendw');

END;
```

In the above example 'DBMS_OUTPUT.PUT_LINE' is a standard api which will PRINT the given input string.

A PL/SQL block example using DECLARATION and EXECUTION section but with out EXCEPTION section :

```
DECLARE

   myvariable  varchar2(100);

BEGIN

   myvariable := 'provendw';

   DBMS_OUTPUT.PUT_LINE('Welcome to - '||myvariable );

END;
```

Welcome to - provendw

In the above example we have declared a variable called 'myvariable' in the Declaration section and assigning value to the variable in Execution section.

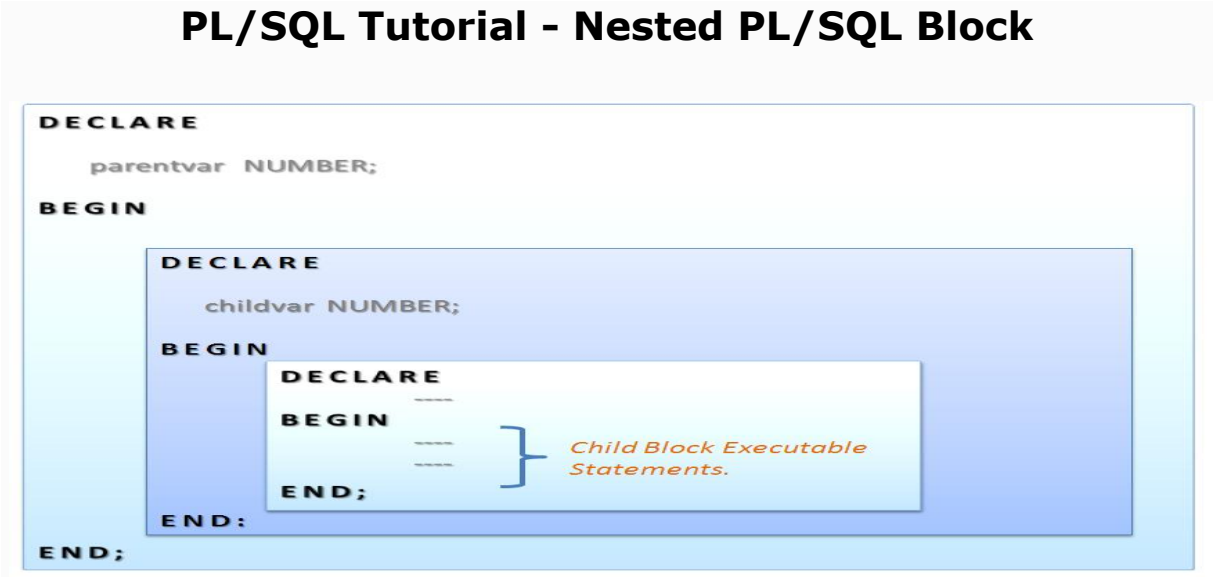
A PL/SQL block example with ALL three sections :

```
DECLARE

   myvariable  varchar2(100);

BEGIN

   myvariable := provendw';
```

```
    DBMS_OUTPUT.PUT_LINE('Welcome to - '||myvariable );
EXCEPTION
  WHEN OTHERS THEN
     DBMS_OUTPUT.PUT_LINE('Exception Raised');
END;
```

Program Output :

Welcome to - provendw



example for Nested Block :

```
DECLARE
    Parentval VARCHAR2(20);
BEGIN
    Parentval := 'First Block';
    dbms_output.put_line(Parentval);
    DECLARE
        childval VARCHAR2(20);
    BEGIN
        childval := 'Second Block';
        DBMS_OUTPUT.PUT_LINE(childval);
    END;
```

```
END;
```

Program Output :

```
First Block
Second Block
```

From the above example we can easily identify the parent and child block, but imagine if we have more PL/SQL blocks one inside another then some times it gets complex to identify which is parent and which is child block.

To overcome this issue and to get better visibility and readability ORACLE has given a provision to provide name to each PL/SQL block. Following is the sample:

'Name of the block' should be specified on top of DECLARE statement. This NAME that we give is just a label that we give to the PL/SQL block, it doesn't hold any logic and doesn't get saved in the database.

```
<<PARENT_BLOCK>>
DECLARE
    Parentval VARCHAR2(20);
BEGIN
    Parentval := 'First Block';
    dbms_output.put_line(Parentval);
    <<CHILD_BLOCK>>
    DECLARE
       childval VARCHAR2(20);
    BEGIN
       childval := 'Second Block';
       DBMS_OUTPUT.PUT_LINE(childval);
       <<GRAND_CHILD_BLOCK>>
     DECLARE
          grand_childval VARCHAR2(20);
     BEGIN
          grand_childval := 'Third Block';
          DBMS_OUTPUT.PUT_LINE(grand_childval);
         END GRAND_CHILD_BLOCK;
    END CHILD_BLOCK;
END PARENT_BLOCK;
/
```

Program Output :

```
First Block
Second Block
Third Block
```

- In the above example, '<<PARENT_BLOCK>>','<<CHILD_BLOCK>>' and '<<GRAND_CHILD_BLOCK>>' are the block names. We use these block names for better readability and to identify the scope of the block better.
- The main advantage of these Nested Blocks is that each block will have its own 'DECLARATION', 'EXECUTION' and 'EXCEPTION' section with which we can have BETTER control on the program logic.

Advantages Of PL/SQL

PL/SQL is a completely portable, high-performance transaction processing language that offers the following advantages:

- ✓ ***Integration with SQL***
- ✓ ***Better Perfrmance***
- ✓ ***Full Portability***
- ✓ ***Tight Security***

Tight Integration with SQL:

- PL/SQL allows us to use SQL data manipulation, cursor control, and transaction control commands, as well as all the SQL functions, operators, and pseudocolumns.
- his extensive SQL support allows us to manipulate Oracle data flexibly and safely.
- PL/SQL fully supports SQL datatypes, reducing the need to convert data which is passed between our applications and the database.

Better Performance:

- A single SQL statement causes two trips on the network
- If multiple SELECT statements are issued, network traffic increases significantly very fast.
- Hence PL/SQL overcomes this problem because entire block of statements can be sent to Oracle at one time, so therefore it reduces the network traffic between the Database and the Application

Full Portability:

- Programs written in PL/SQL can run in any Operating System whereever our oracle DataBase got installed.
- With PL/SQL we can write portable programs which we can reuse in different environments.

Tight Security:

- While moving PL/SQL stored procedures from client to the server we can protect out programs from network tampering, and also we can restrict to the access of programs.
- Triggers written in PL/SQL can obey the business rules for the changes which we made in database.

PL/SQL is nothing but a extension to **Structured Query Language** with **Procedural Concepts** and below list explains how PL/SQL supports :

- PL/SQL supports to '**execute a block of statements as unit'.**
- PL/SQL supports '**variables and constants'.**
- PL/SQL supports **'conditional constructs (IF)'.**
- PL/SQL supports **'iteration control statements (loops)'.**
- PL/SQL supports **'error handling using exceptions'.**
- PL/SQL supports to '**define composite data types'.**
- PL/SQL supports to '**execute a block of statements automatically based on the event using database triggers'**.
- PL/SQL supports to '**store and share the code using sub-programs'.**
- PL/SQL supports '**Object Oriented Programming**'.
- PL/SQL supports '**Web Applications and Pages'.**

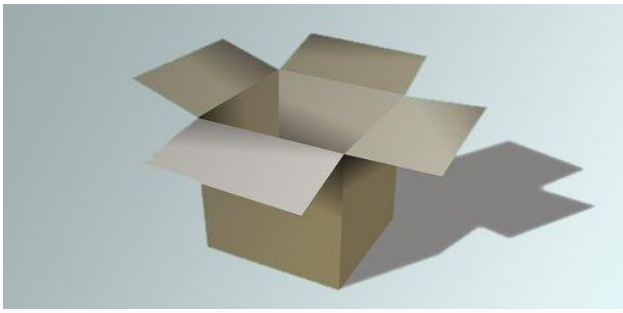
PL/SQL Tutorial - What are Variables?

What are Variables?

In any programming language (not just in PL/SQL) VARIABLES play a vital role in building program logic. We cannot imagine a code with out defining variables.

What is a Variable ?:

- Variables are like placeholders or temporary storage locations in database, we use them to store/manipulate data at the time of PL/SQL block execution.
- Variables can store the values that can change through the PL/SQL Block.



- We build our program logic by manipulating and assigning data to the variables, we will change them as on when it is required in the program.
- We need to identify the variables upfront before we write our code in execution section.
- Variables are always defined in the declaration section and we use them in execution and exception sections
- During declaration of the variable in declaration section we should foresee what kind of data we wanted to store in the variable and accordingly we should specify the 'Data Type'.
- Data Type is something that tells us what kind of data should variable store.

Syntax to Declare a Variable

```
DECLARE
    <variable Name>  <datatype>;
    --
    --
```

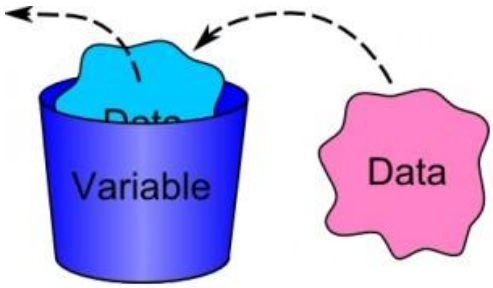
Syntax to Assign a value to the variable

```
DECLARE
   <variable Name>  <datatype>;
   --
   --
BEGIN
   <variable name> := <value>;
   --
   --
END;
```

Syntax to Declare a Variable and Assign value at the same time

```
DECLARE
   <variable Name>  <datatype> := <value>;
   --
```

We can change the value of the variable any number of times in the program logic to fetch the desired result.



Below are the different ways of creating a variable :

```
DECLARE
    a NUMBER;
    b NUMBER := 5;
    c NUMBER DEFAULT 6;
    d CONSTANT NUMBER(3):=87;
    e NUMBER(2) := 10;
    f BOOLEAN := TRUE;
    ---
```

In the above example 'a,b,c,d,e,f' are variables , 'NUMBER, BOOLEAN' are datatypes and 'CONSTANT, DEFAULT' are system keywords.

CONSTANT :

when we specify 'CONSTANT' keyword in the variable declaration then that variable will hold same value through out the program.

DEFAULT :

Defualt is a system keyword and value specified along with it will be the initial value in the variable. It is also same as assigning a value during the declaration.

```
var NUMBER DEFAULT 6;
  (OR)
var NUMBER := 6;
```

PL/SQL Data Types

- Every PL/SQL variable has a specific type associated with it describing the data stored in the variable. Based on the type and usage, Data types are categorized into four types :
- Every PL/SQL variable has a specific type associated with it describing the data stored in the variable. Based on the type and usage, Data types are categorized into four types :

Scalar data type

Scalar datatypes are the same data types that we use in SQL. These can be used in PL/SQL to define variables.

Reference data type

We use reference datatypes when we wanted to refer to the type of some database column.

Composite data type

Composite data types are part of advanced PL/SQL, we will be learning them in coming chapters.

LOB data types.

We use LOB datatypes when we wanted to store large object information like images, vidoes…..etc

Scalar datatypes are used to classify the type of data that is being stored in the variable.

- Scalar Data Type holds a single value.
- Scalar Data Types are the same data types that are used in SQL for defining column types for oracle database tables.
- Scalar Data Types supports Boolean variables.

Scalar Data Types:

Data type	**Description**	**Sample Declaration**
Varchar2	Variable-length character string	Lastname varchar2(30)
char	Fixed-length character string	Gender char(1)
Number	Floating-point, fixed-point, or integer number	Price number(5)
date	Date and time	Hire_date Date;

Examples for Scalar Data Types:-

```
DECLARE
    name       VARCHAR2(10):='SMITH';
    empno       NUMBER;
    hiredate    DATE;
    ---
```

- In the above example, **'name'** is character variable and it holds a single value
- **'empno'** is a scalar variable which stores numeric values.
- **'hiredate'** is a scalar variable which stores date values.

Reference Data Types

We use REFERENCE DATA TYPES when we wanted to refer or copy the data type of an existing column/record in a database table. The advantage of these reference types is that we can hide the actual data type of the variable.

There are two types of Reference Types :

- **%TYPE** – '%TYPE' is used to copy the data type of a column in a database table.
- **%ROWTYPE** – 'ROW%TYPE' is used to copy the structure of entire record in a database table. We will discussing more about %ROWTYPE in Record using %ROWTYPE lesson

- Using **%TYPE** we declare scalar variables in the pl/sql that refer to a particular column of a existing database table or view or synonym.

Syntax for %TYPE :-

```
DECLARE
    <variable name>   <table name>.<column name>%TYPE;
    ---
    ---
```

Example for %TYPE :-

```
DECLARE
  v_deptno   dept.deptno%TYPE;
  v_dname    dept.dname%TYPE;
BEGIN
  SELECT deptno, dname
    INTO v_deptno, v_dname
    FROM dept
   WHERE deptno = 10;
  DBMS_OUTPUT.PUT_LINE (v_deptno || '  '|| v_dname);
END;
```

Output:

10 ACCOUNTING

LOB Data Types

A LOB is a data type that is used to store large, unstructured data such as 'text',' graphic images',' video clippings', and so on.

Different LOB Data Types:

BLOB

Binary Large Object – column or variable of type BLOB can store up to 4GB of binary data in each record.

CLOB

Character Large Object – column or variable of type CLOB can store up to 4GB of character data in each record.

BFILE

It can store a file of size 4GB externally outside database for each record and can refer to that from inside the database.

There are two distinct parts of LOB in order to access it :

- **LOB value**: the data that constitutes the real object being stored.

LOB Locator: A pointer to the location of the LOB value stored in the database.

There is separate article for 'LOB DATATYPES' example in PLSQL Article section, as it is a advanced topic.

Constant Variables

- A **CONSTANT VARIABLE** is a variable that is defined in the declaration section of the block which remains unchanged throughout the program. To define a CONSTANT variable we need to explicitly specify a keyword called 'CONSTANT' when declaring the variable.

```
DECLARE
    <variable name>  CONSTANT <data type>  :=  <value>;
    ---
    ---
```

- We define constant variable only when we want to make sure that data in the variable should be unchanged irrespective of the program logic.
- For example: As per policy if we want to apply 5% tax on all the invoices irrespective of customer then in customer package we can declare a constant and assign a value – 5%.

```
DECLARE
    v_tax CONSTANT number (3):= 5;
    ---
    ---
```

- We should assign a value to a constant when we declare it. If we do not assign a value when declaring it and try to assign a value in the execution section then program will result in error.

Example: Below code will result in error as assignment made to the variable in execution section.

```
DECLARE
    /* Declaring constant variable */
    v_tax CONSTANT number (3):= 5;
BEGIN
    v_tax := 10;
    /* System will raise an error as we are trying to assign a different value other than constant */
    DBMS_OUTPUT.PUT_LINE (v_tax);
END;
```

Output:
PLS-00363: expression 'V_TAX' cannot be used as an assignment target.

Conclusion :

Once we declare a variable with constant then we can not change the value through out the program execution. If we try to change the value then system will raise an error.

Scope Of Variables

- The scope of a variable is the portion of the program in which the variable can be accessed. When a variable goes out of scope, thePL/SQL engine will free the memory used to store the variable.

Based on their type of declaration we can classify variables into two types.

- ✓ **Local variables** – Local variables are declared and can be accessed in a specific block.
- ✓ **Global variables** – These are declared in a parent block and can be referenced by itself an

For Example:

- In the below code we are creating two variables 'v_num1' and 'v_num2' in the parent block and assigning their sum to the variable 'v_sum' created in the child block. But 'v_sum' cannot be accessed in the parent block. The variables 'v_num1' and 'v_num2' can be accessed anywhere in the block.

```
DECLARE
   v_num1 NUMBER;
```

```
    v_num2 NUMBER;
BEGIN
                                        /* Outer Block */
      v_num1:= 100;
      v_num2:= 200;
         DECLARE                                  /* Inner Block */
             v_sum NUMBER;
         BEGIN
             v_sum:= v_num1 + v_num2;
         END;                            /* End of Inner Block */
END;
```

PL/SQL procedure successfully completed.

Now lets look at a Negative Scenario : Access v_sum in the parent block

```
DECLARE
    v_num1 NUMBER;
    v_num2 NUMBER;
BEGIN                                              /* Outer Block */
      v_num1:= 100;
      v_num2:= 200;
         DECLARE                                  /* Inner Block */
             v_sum NUMBER;

 BEGIN
             v_sum:= v_num1 + v_num2;
         END;                        /* End of Inner Block */
         /*Access child variable v_sum in the parent block*/
         DBMS_OUTPUT.PUT_LINE(v_sum);
END;                                  /* End of Outer Block */
```

Error at line 1

PLS-00201: identifier 'V_SUM' must be declared

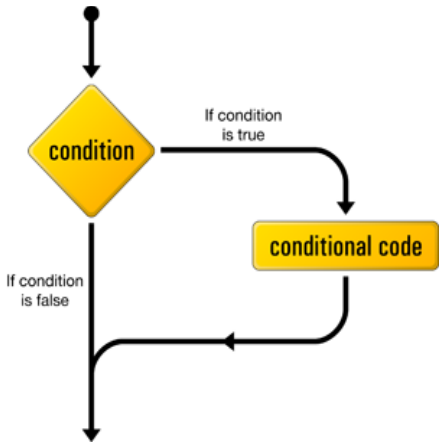
PL/SQL: Statement ignored

PL/SQL Tutorial - Conditional Statements

Conditional Statements:

Conditional Statements – is another important concept in **PL/SQL** which we will be using very frequently to achieve our business logic.
A **conditional statement** refers to the ability to process a portion of code depending on whether certain criteria is met or not.
i.e., In a program, if a condition **is met** then we wanted to execute certain portion of code and if condition **is not met** then we wanted to execute different portion of code.

Using conditional statements in pl/sql gives us better programming control over the code.



Below are different types of Conditional Statements in PL/SQL :

- **IF STATEMENT**
- **IF – ELSE.**
- **NESTED IF.**
- **ELSIF.**
- **CASE STATEMENT**

Based on Variable.

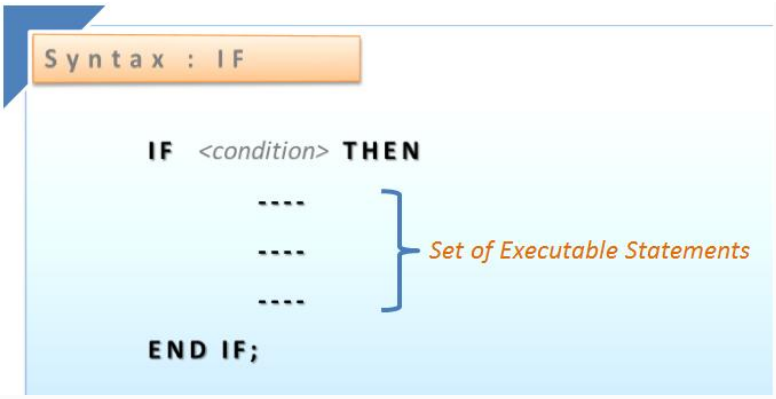
Based on Condition.

Conditional Statements is a **programming concept**, Just like in PL/SQL we can find these in all other programming languages as well but in other programming languages SYNTAX and USAGE will be different.

ONLY IF STATEMENT

- IF statement allows us to execute a sequence of statements based on a condition.
- Condition specified in IF statement should evaluate to either TRUE/FALSE and that condition can be a single expression or a combination of multiple expressions. We can even directly use a Boolean variable in if statement as a condition.
- This is the most basic conditional evaluation statement. i.e. IF a condition is met, THEN do something.
- IF statement must be terminated with a corresponding END IF clause.

- and IF a condition is not met, skip the code that follows till 'END IF' and continue with the rest of the program.



IF Statement Example with 'TRUE' condition :-

```
DECLARE
    a  NUMBER := 2;
    b  NUMBER := 2;
BEGIN
   IF a=b THEN
       DBMS_OUTPUT.PUT_LINE ('We are in IF Block');
   END IF;
DBMS_OUTPUT.PUT_LINE ('Outside IF Statement');
END;
```

Output:-

```
We are in IF Block

Outside IF Statement
```

In the above example, condition 'a=b' evaluated to **TRUE** as 'a' value is '2' and 'b' value is also '2' so the code inside IF statement got executed.

IF Statement Example with 'FALSE' condition :-

```
DECLARE
    a  NUMBER := 2;
    b  NUMBER := 3;
BEGIN
   IF a=b THEN
       DBMS_OUTPUT.PUT_LINE ('We are in IF Block');
   END IF;
DBMS_OUTPUT.PUT_LINE ('Outside IF Statement');
END;
```

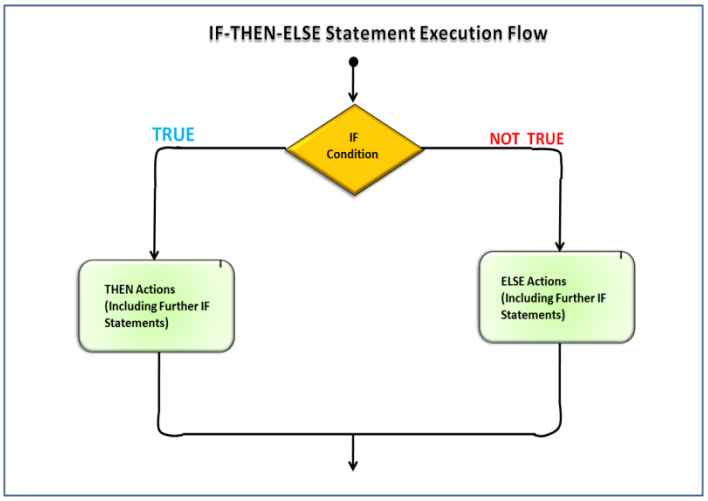
Output:-

```
Outside IF Statement
```

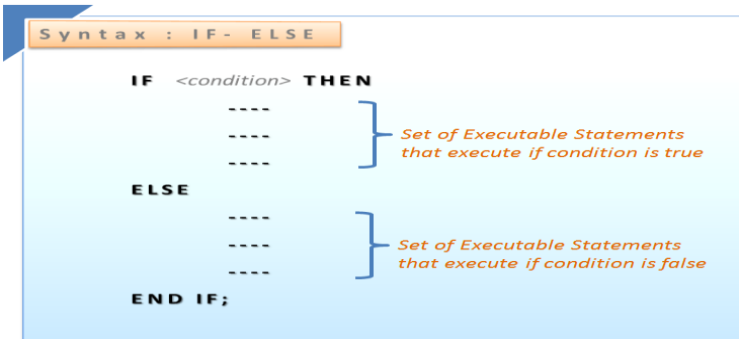
In the above example, condition 'a=b' evaluated to **FALSE** so the code inside IF statement didn't get executed and the program control executed the code outside IF statement.

IF - ELSE Statement

IF- ELSE statement is a continuation to **IF** statement with additional ELSE block.
IF-ELSE statement enables us to specify two different groups of statements for Execution. i.e. One group is evaluated when the condition evaluates to "TRUE", the next group is evaluated when the condition evaluates to" FALSE".



Below is the syntax for **IF- ELSE** statement. The additional keyword in SYNTAX that we can observe with respect to only 'IF Statement' is the 'ELSE' command. The else group of statements starts from 'ELSE' command to 'END IF' command.




```
DECLARE
    a  NUMBER := 2;
    b  NUMBER := 3;
BEGIN
   IF a=b THEN
       DBMS_OUTPUT.PUT_LINE ('We are in IF Block');
   ELSE
       DBMS_OUTPUT.PUT_LINE ('We are in ELSE Block');
   END IF;
END;
```

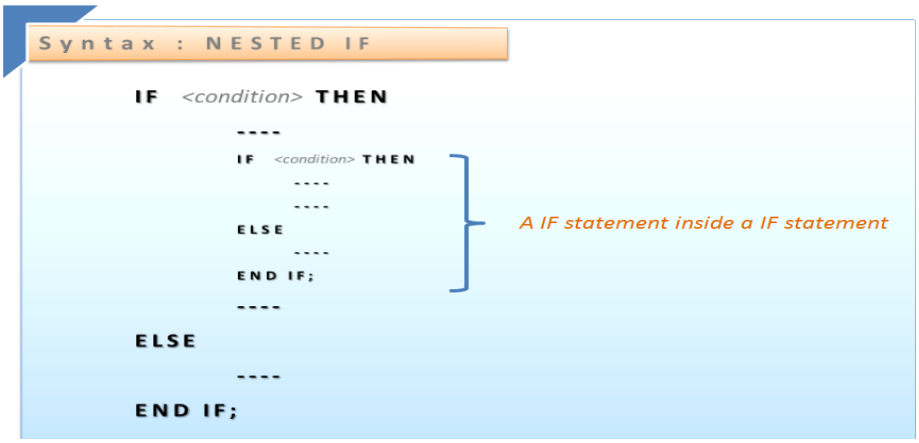
o/p:

```
We are in ELSE Block
```

In the above example, the condition 'a=b' evaluated to false as 'a' value is '2' and 'b' value is '3', there fore code in 'ELSE' block is executed.

NESTED IF Statement

- Calling conditional IF statement one inside another is called as Nested IF statement
- In Nested IF, result of the first IF statement will decide the execution of child IF statements
- Each nested IF statement must be terminated with a corresponding END IF clause.
- We can specify any number of NESTED if statements.
- For NESTED IF statement, all the If statements will be executed in hierarchical / top-down approach. i.e. first parent IF statements will be executed and if condition is TRUE then execution will proceed to the child IF statements.



Example:-

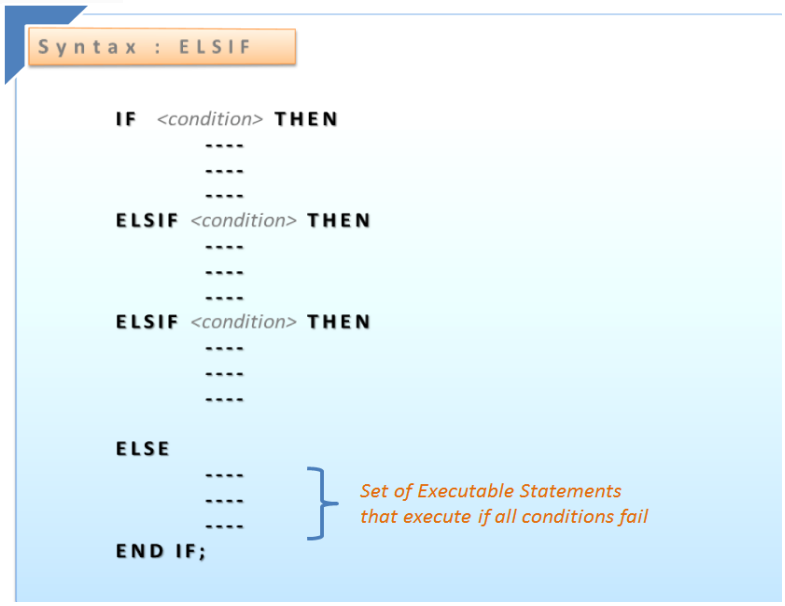
```
DECLARE
  v_deptno   NUMBER (2) ;
 v_name     VARCHAR2 (100) := 'SALES' ;
BEGIN
  SELECT deptno
  INTO   v_deptno
  FROM   dept
 WHERE   dname = v_name;
 DBMS_OUTPUT.PUT_LINE ('Department Number fetched is - '||v_deptno);
 IF (v_deptno > 20 )     THEN
    DBMS_OUTPUT.PUT_LINE ('Department Number is greater than 20');
    IF (v_deptno = 30) THEN
       DBMS_OUTPUT.PUT_LINE ('We are in Department Number 30');
    ELSE
        IF (v_deptno = 40) THEN
           DBMS_OUTPUT.PUT_LINE ('We are in Department Number 40');
        END IF;
    END IF;
 ELSE
    DBMS_OUTPUT.PUT_LINE ('Department Number is less than or equal to 20');
    IF (v_deptno = 20) THEN
       DBMS_OUTPUT.PUT_LINE ('We are in Department Number 20');
    ELSE
        IF (v_deptno = 10) THEN
           DBMS_OUTPUT.PUT_LINE ('We are in Department Number 10');
        END IF;
    END IF;
 END IF;
END;
```

Output:-
Department Number fetched is - 30
Department Number is greater than 20
We are in Department Number 30

- Deptno for Deptname'SALES' is '30'. In the above example, We have multiple IF statements one in another. i.e. based on the 'Deptno' program evaluates first IF condition i.e '> 20' and proceeding with the execution of child IF statements.
- ELSIF Statement
- ELSIF condition statement is very similar to NESTED IF, but in ELSIF we specify all IF statements in a sequential way using 'ELSIF' keyword.

- 'ELSIF' is the system keyword (i.e. it means both ELSE and IF)
- 'ELSIF' works in a sequential way. i.e If the first condition is false or null, then ELSIF clause tests another condition.
- Conditions are evaluated – one by one from top to bottom.
- An If statement can have any number of ELSIF clauses. The final ELSE clause is optional.
- ELSIF statement is useful when we have multiple conditional statements to evaluate.

Please find the syntax below



If any of the condition is true then its associated sequence of statements are executed and program control jumps to the end of IF statement. If all the conditions are false then statements in ELSE clause will be executed.

Example:-

```
DECLARE
  v_deptno   NUMBER (2) ;
  v_name     VARCHAR2 (100) := 'SALES' ;
BEGIN
  SELECT deptno
  INTO  v_deptno
  FROM   dept
  WHERE   dname = v_name;
  DBMS_OUTPUT.PUT_LINE ('Department Number fetched is - '||v_deptno);
  IF (v_deptno = 10)
```

```
  THEN
     DBMS_OUTPUT.PUT_LINE ('We are in Department Number 10');
    ELSIF (v_deptno = 20)
    THEN
      DBMS_OUTPUT.PUT_LINE ('We are in Department Number 20');
     ELSIF (v_deptno = 30)
     THEN
        DBMS_OUTPUT.PUT_LINE ('We are in Department Number 30');
      ELSIF (v_deptno = 40)
      THEN
         DBMS_OUTPUT.PUT_LINE ('We are in Department Number 40');
      ELSE
         DBMS_OUTPUT.PUT_LINE ('Sorry! No such department exists');
  END IF;
END;
```

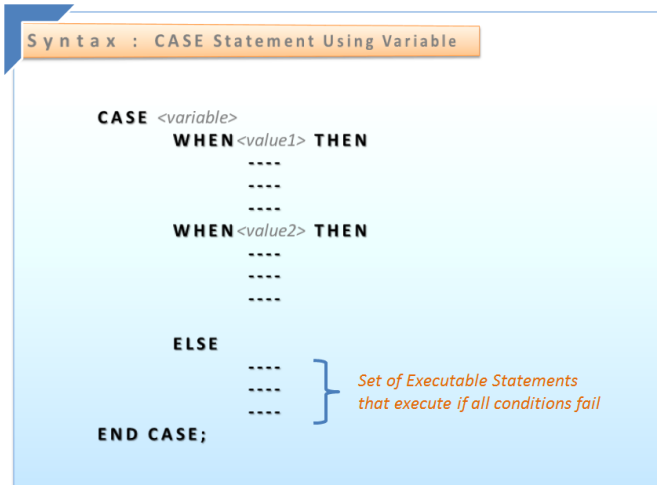
Output:-

We are in Department Number 30

- Deptno for Deptname 'SALES' is '30'. In the above example, multiple ELSIF statements are evaluated and finally condition 'v_deptno = 30' results TRUE executing 'DBMS_OUTPUT.PUT_LINE ('We are in Department Number 30');'.
- with this example we can see a clear difference between NESTED IF and ELSIF.
- CASE Statement Based On Variable

Based on the Variable:

- The PL/SQL CASE statement is another type of conditional statement which allows us to execute a sequence of statements based on a selector.
- A selector can be anything such as variable, function, or expression that the CASE statement evaluates to some value.
- The selector in CASE statement is followed by one or more WHEN clauses, which are checked sequentially.
- The value of the selector determines which clause is executed. If the value of the selector equals the value of the WHEN clause expression, that WHEN clause is executed.



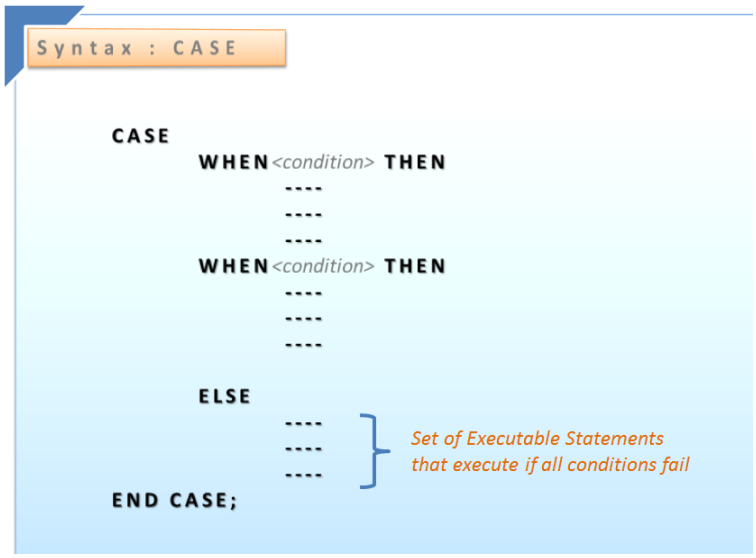
The reserved word "CASE" marks the beginning of the CASE statement.
The selector is a value that determines, which "WHEN" clause should be executed.

Example:-

```
DECLARE
  v_deptno NUMBER (2):=20;
  v_dname VARCHAR2 (100);
BEGIN
 SELECT dname INTO v_dname
 FROM dept WHERE deptno = v_deptno;
/* Here we try to specify the case for a variable v_deptno. */
CASE V_deptno
 WHEN 10 THEN
     DBMS_OUTPUT.PUT_LINE ('The Department Name of Deptno is: '||v_dname);
 WHEN 20 THEN
     DBMS_OUTPUT.PUT_LINE ('The Department Name of Deptno is: '||v_dname);
 WHEN 30 THEN
     DBMS_OUTPUT.PUT_LINE ('The Department Name of Deptno is: '||v_dname);
 WHEN 40 THEN
     DBMS_OUTPUT.PUT_LINE ('The Department Name of Deptno is: '||v_dname);
 ELSE
    DBMS_OUTPUT.PUT_LINE ('Sorry ! No Such Records Exists');
 END CASE;
END;
```

Output:-
The Department Name of Deptno is: **RESEARCH**

CASE Statement Based on Condition
Based on the condition:
In this type we do not use selector, instead we use when conditions directly. Here condition is a 'expression' which will evaluate to Boolean value.
When a particular search condition evaluates to true, the group of statements associated with this condition are executed.



```
DECLARE
  v_deptno  NUMBER (2):=30; /* Declaring and assigning a value to variable */
  v_dname   VARCHAR2 (100);
BEGIN
  SELECT dname INTO v_dname
  FROM dept WHERE deptno = v_deptno;

 /* Giving condition to SQL statement in where clause, here we are checking with the variable v_deptno for
which we already assigned value. */
/* Here we directly specify the condition in when clause using variable name. */

CASE
    WHEN v_deptno = 10 THEN
      DBMS_OUTPUT.PUT_LINE ('Department Name for a defined variable is : '||v_dname);
    WHEN v_deptno= 20 THEN
      DBMS_OUTPUT.PUT_LINE ('Department Name for a defined variable is : '||v_dname);
    WHEN v_deptno= 30 THEN
      DBMS_OUTPUT.PUT_LINE ('Department Name for a defined variable is : '||v_dname);
    WHEN v_deptno= 40 THEN
      DBMS_OUTPUT.PUT_LINE ('Department Name for a defined variable is : '||v_dname);
    ELSE
      DBMS_OUTPUT.PUT_LINE ('Sorry! No Such records exists');
  END CASE;
END;
```

O/P:

Department Name for a defined variable is : **SALES**

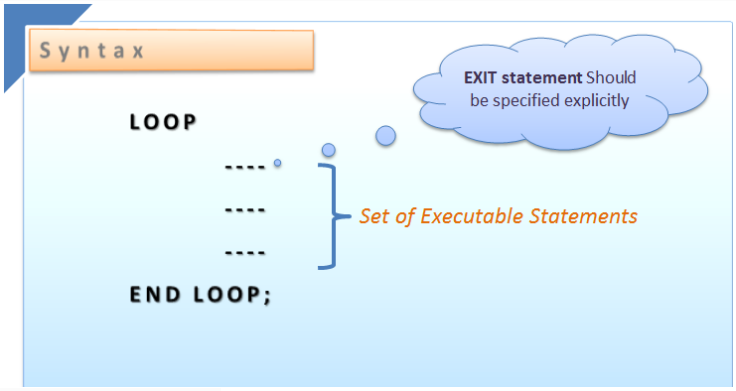
PL/SQL Tutorial - Iteration Control Statements (LOOP)

Iteration Control Statements (LOOP)

- An iterative control Statements are used when we wanted to execute same set of statements for a specified number of times.
  i.e., it will execute a set of statements repeatedly until a specified condition is TRUE.
- There are 4 types of Iteration Control Statements:

1) Simple loop
2) While loop
3) Numeric for loop
4) Cursor for loop

Simple loop- It is an infinite loop task A Simple Loop is used when we wanted to execute the set of statements at least once before the loop terminates. An EXIT condition must be explicitly specified in the loop to fetch the control out of the loop. Otherwise the loop will get into an infinite number of iterations.



- To break the simple loop
  Syntax:-

```
EXIT WHEN <condition>;
IF <condition> THEN
EXIT;
END IF;
```

- Standard steps to be followed while using Simple Loop.

  1) Initialize a variable before the loop begins.
  2) Increment the variable in the loop.
  3) EXIT WHEN statement is used to exit from the Loop.

Example:-

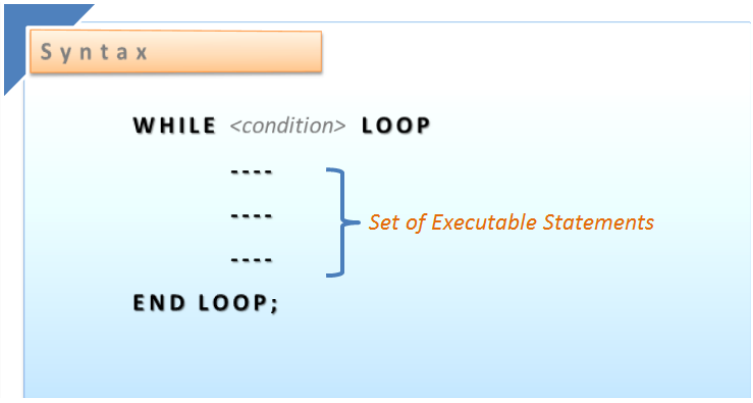
```
DECLARE
v_num number (10):=0;
BEGIN
  DBMS_OUTPUT.put_line ('the numbers are--');
  /*entering into loop which fetches one by one value*/
  LOOP /* Declaration of loop statement */
  v_num:=v_num+1;   /*incrementing one value*/
  DBMS_OUTPUT.PUT_LINE(v_num); /* Displaying the value */
  EXIT WHEN (v_num=10);
```

```
/* Checking the condition, if v_num is equals 10 cursor exits and goes to END LOOP otherwise it will
execute from LOOP statement again */
  END LOOP;
END;
```

Output-
1 2 3 4 5 6 7 8 9 10

While Loop Statement

WHILELOOP can be used when we wanted to execute set of statements until condition is true. This condition is evaluated at the beginning of each iteration. The execution of statements continues until the condition becomes false.



Example:-

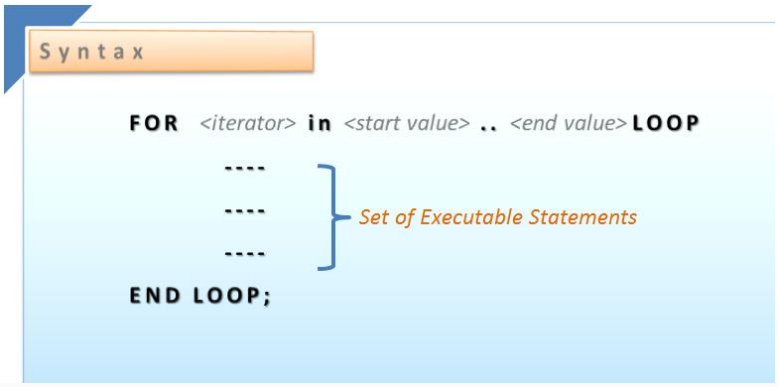
```
DECLARE
   v_num number (10):=0;
BEGIN
   /* Before entering into the WHILE loop oracle checks whether the condition is satisfied or not */
   WHILE (v_num <=10) loop
   DBMS_OUTPUT.put_line (v_num);
   v_num:= v_num +1; /* Incrementing Variable value */
   /* Note that if we are not increment variable value then loop will never exit */
   END LOOP;
END;
```

O/P:
1 2 3 4 5 6 7 8 9 10

For Loop Statement

FOR LOOP can be used when we wanted to execute a set of statements for a specific number of times. Iteration occurs between the given start and end integer values.

- The counter is always incremented by '1'.
- The loop exits when the counter reaches the value of the end integer



- Reverse (optional) – accepts the value in reverse order.
- '..' these two dots are considered as increment/range operator between lower bound and higher bound àit increments the variable by 1 always.

Example:-

```
BEGIN
  DBMS_OUTPUT.put_line (' The Numbers Are');
  DBMS_OUTPUT.put_line ('-------------------------');
  FOR v_num IN 1..10
  LOOP
    DBMS_OUTPUT.put_line (v_num);
  END LOOP;
END;
```

o/p:
the numbers are : 1 2 3 4 5 6 7 8 9 10

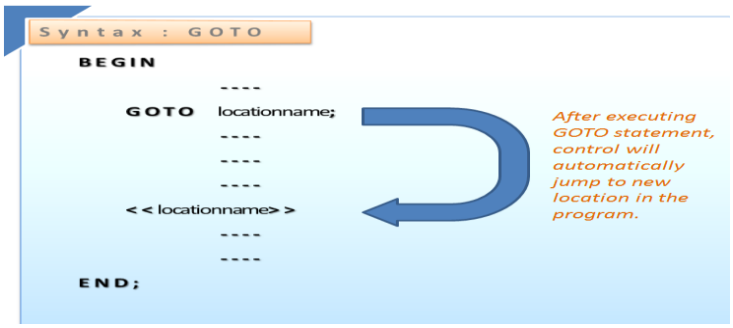
The following example shows the REVERSE order of FOR loop

```
BEGIN
  DBMS_OUTPUT.put_line (' The Numbers Are');
  DBMS_OUTPUT.put_line ('-------------------------');
  FOR v_num IN REVERSE 1..10
  LOOP
    DBMS_OUTPUT.put_line (v_num);
  END LOOP;
END;
```

o/p:
reverse order:10 9 8 7 6 5 4 3 2 1

GOTO And LABELS

GOTO statement is used when we want to skip a specific set of statements for execution. GOTO statement pulls the program control from the current position and starts executing the statements from <<label>> defined position.



Where *label* is a label defined in the PL/SQL block. Labels are enclosed in double angle brackets. When a goto statement is evaluated, control immediately passes to the statement identified by the label.

Example:
/*here we are going to know how GOTO is going to work and printing the numbers from 1 to 5*/

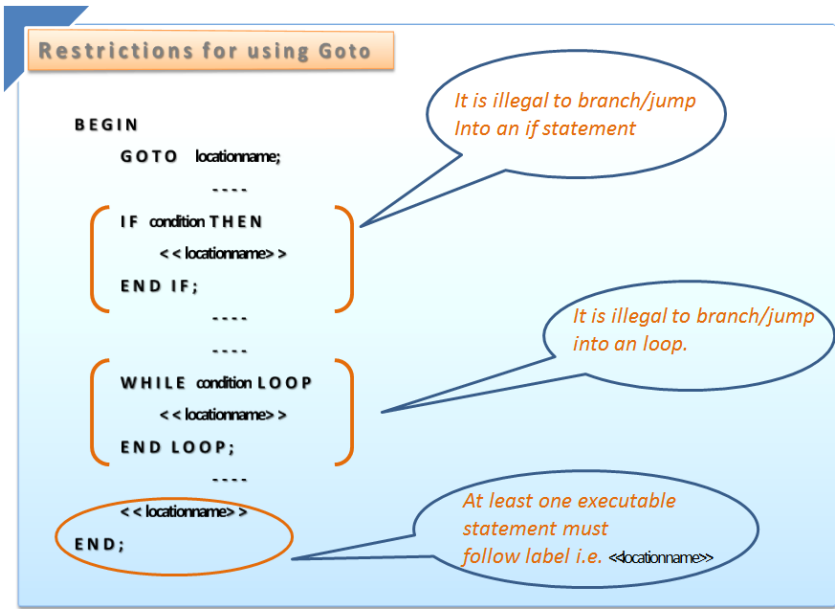
```
BEGIN
   FOR v_num IN 1..5
   LOOP
   DBMS_OUTPUT.put_line (v_num);
   /* when ever it reaches its maximum value then it directly go to exit loop*/
    IF v_num = 3
     THEN
       GOTO exit_loop;
       /* Here we are giving a condition to exit from loop giving some IF condition, it means if the condition
specified in the IF condition is satisfied then cursor will exit from loop without waiting the call of FOR loop */
    END IF;
  END LOOP;
  <<exit_loop>>
 /* it is an executable statement which should be mandatory*/
  DBMS_OUTPUT.put_line (' End of program');
```

O/P:
1 2 3 end of the program

Restrictions for using Goto

We cannot use GOTO statement where ever we want in PLSQL code, there are several restrictions defined for the usage as GOTO jumps the program control from one place to another.

- It is illegal to branch into an inner block, loop.
- At least one executable statement must follow.



Example:-

```
BEGIN
  FOR v_num IN 1..5
  LOOP
    DBMS_OUTPUT.put_line (' v_num = ' || v_num);
    /* when ever it reaches its maximum value then it directly go to exit loop*/
    IF v_num = 4
    THEN
      GOTO exit_loop;
    END IF;
  END LOOP;
  <<exit_loop>>
  DBMS_OUTPUT.put_line (' End of program');
  /* it is an executable statement which should be mandatory*/
END;
```

O/P:

```
v_num = 1
v_num = 2
v_num = 3
v_num = 4
End of program
PL/SQL procedure successfully completed.
```

PL/SQL Tutorial - Cursors

Cursors

- A cursor can be defined as a temporary work area created in the system memory whenever a SQL statement is executed.
- CURSOR is a handle or pointer to the context area. Using a cursor, the PL/SQL program can control the context area.
- To process SQL statements oracle needs to create a area of memory known as the CONTEXT AREA or PROCESS GLOBAL AREA (PGA).
- Each cursor contains information on a select statement and access to the rows fetched by the select statement.
- A cursor can hold multiple rows retuned by the select statement, but can process only one row at a time. The set of rows that cursor holds is called the active set.

There are two types of cursors in PL/SQL:

- Implicit Cursors
- Explicit Cursors

Implicit Cursors:

Implicit cursor is automatically created by the system whenever program executes DML statements like DELETE, INSERT, UPDATE and SELECTS statements.

Explicit Cursors:

Explicit cursor should be explicitly defined in the declaration section of the PL/SQL Block. Explicit cursor is created on a SELECT Statement which returns more than one row from the database.

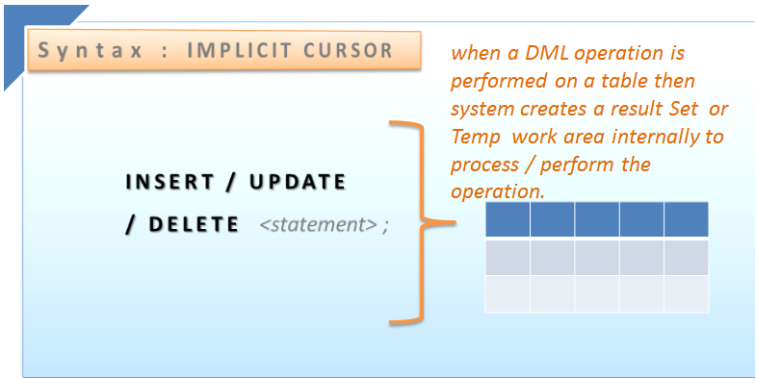
Example:

```
CURSOR c_dept IS
```

```
SELECT *
FROM EMP
WHERE deptno = 10;
```

Implicit Cursor

- Implicit cursor is automatically defined by Oracle when DML operations are performed by user.
- Implicit cursor has fixed name 'SQL'.
- Implicit cursor gives the status of 'DML' statements in PL/SQL block.



- Implicit cursor will not support cursor operations like **open, fetch, and close**.
- Implicit cursor supports **%found**, **%notfound**, **%rowcount** attributes.
  - **%found** returns true if DML is success.
  - **%notfound** returns TRUE if DML is failed.
  - **%rowcount** returns number of rows manipulated by DML statement.
- **%isopen** is not valid.
  - Oracle provides few attributes known as implicit cursor attributes to know the status of DML operations.
  - These cursor attributes tells us how many rows/records are affected for the DML operation.

  Following are the **implicit cursor attributes** available:
- **SQL%found**
  Returns TRUE if operation has affected at least one row in the table else will return FALSE.
- **SQL%notfound**
  Returns FALSE if operation has affected at least one row in the table else will return TRUE.
- **SQL%rowcount**
  Returns number of rows affected.

EXAMPLE:

```
BEGIN
/*here we are going to perform DML (update) operation. When we perform any DML operation IMPLICIT
CURSOR will automatically activate*/
  UPDATE emp
  SET sal = sal + sal * .02;
  DBMS_OUTPUT.put_line ('emp table updated');
END;
```

Output:-

```
emp table updated
```

Example 2:-

```
DECLARE
 v_dept  VARCHAR2(20) := 'SALES';
```

```
BEGIN
 /*here we are going to update EMP table if the specified Department Name exists .*/
   UPDATE emp
     SET sal = sal + sal * .02
     WHERE deptno = (SELECT deptno FROM DEPT WHERE dname=v_dept);
/*here we are going to trap update statement status*/
   IF SQL%FOUND
   THEN
/*here we get the count, of how many records are going to update if there is 'department name which we
provided'*/
   DBMS_OUTPUT.put_line ( SQL%ROWCOUNT|| '  '|| 'employee(s) are updated successfully');
   ELSE
   DBMS_OUTPUT.put_line ('no such Department Name exists with ->' || v_dept);
   COMMIT;  /* Committing the DataBase to made the changes permanent */
   END IF;
END;
```

O/P:

```
6  employee(s) are updated successfully
```

Let us consider for example we are giving the Department Name which does not exists, then the above program uses of SQL%NOTFOUND.

```
DECLARE
  v_dept  VARCHAR2(20) := 'MARKETING';
BEGIN
 /*here we are going to update EMP table if Department Name exists.*/
UPDATE emp
    SET sal = sal + sal * .02
    WHERE deptno = (SELECT deptno FROM DEPT WHERE dname=v_dept);
/*here we are going to trap update statement status*/
   IF SQL%FOUND
   THEN
   /*here we get the count, of how many records are going to update if there is 'department name which we
provided'*/
   DBMS_OUTPUT.put_line ( SQL%ROWCOUNT|| '  '|| 'employee(s) are updated successfully');
   ELSE
   DBMS_OUTPUT.put_line ('no such Department Name exists with ->' || v_dept);
   COMMIT;  /* Committing the DataBase to made the changes permanent */
   END IF;
END;
```

O/P:

```
no such Department Name exists with ->MARKETING  /*DBMS Output */
```

Example 3:- Implicit Cursor for DELETE command

```
Declare
   v_dept VARCHAR2(20):='RESEARCH';
Begin
/*here we are going to delete records from EMP table*/
DELETE FROM emp WHERE deptno=deptno = (SELECT deptno FROM DEPT WHERE dname=v_dept);
/*here we are going to count the employees in each deptno. If, in a deptno there are employees greater than
3 members then we are going to stop the DML statement otherwise delete the employees*/
```

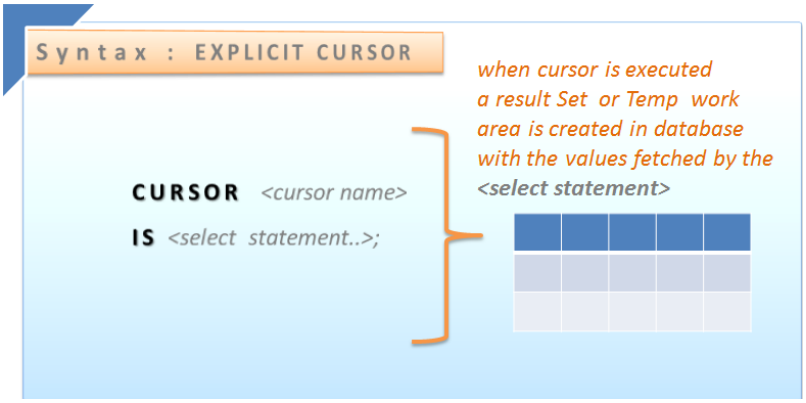
```
 If SQL%ROWCOUNT>=3 THEN
      DBMS_OUTPUT.PUT_LINE ('Cannot delete 3 or more than 3 employees at a time.');
      ELSE
      DBMS_OUTPUT.put_line(SQL%rowcount||' '||'employees are deleted');
     COMMIT; /* committing the Data Base to made the changes permanent */
 END IF;
END;
/*Note that if we give the Department Name which has employees of less than 3 then it will delete*/
```

Output:-

Cannot delete 3 or more than 3 employees at a time.

Explicit Cursor

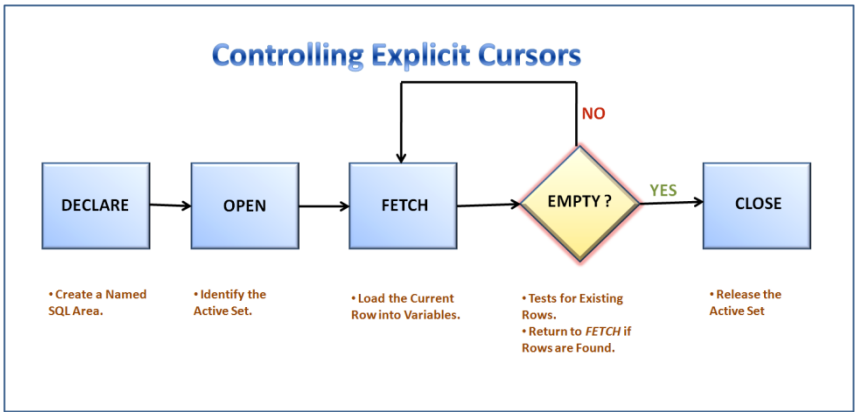
Explicit cursor should be explicitly defined in the declaration section of the PL/SQL Block. Explicit cursor is created on a SELECT Statement which returns more than one row from the database. Explicit cursor must be defined before the variable declarations in the program.



Explicit cursors provide control over cursor processing that is not possible with other types of cursors.
They are meant to work with SELECT statements that return more than one record at a time.
To use an explicit cursor, it must be declared, opened, fetched from, and closed.

Explicit Cursor Functions:-

- Can process beyond the first row returned by the query, row by row.
- Keep track of which row is currently being processed
- Allow the programmer to manually control explicit cursors in the pl/sql block.

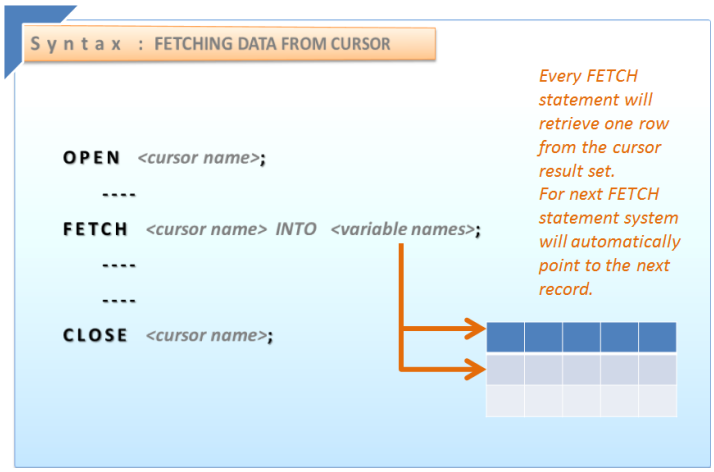


Controlling Explicit Cursors:-

1. Declare the cursor by naming it and defining the structure of the query to be performed with in it.

2. Open the cursor. The OPEN statement executes the query and binds any variables that are referenced. Rows identified by query are called the active set and are now available for fetching.
3. Fetch data from the cursor. In the flow diagram shown on the slide, after each fetch we test the cursor for any existing cursor. If there are no more rows process, then we must close the cursor.
4. Close the cursor. The CLOSE statement releases the active set of rows. It is now possible to open the cursor establish a fresh active set

We use the OPEN, FETCH, and CLOSE statements to control a cursor. The OPEN statement executes the query associated with the cursor, identifies the result set, and positions the cursor before the first row.
The FETCH statement retrieves the current row and advances the cursor to the next row. When the last row has been processed, the CLOSE statement disables the cursor.



CURSOR STAGES
Following are different stages to execute a cursor:

- Open <cursor name>; Is used to open the cursor.
- Fetch <cursor name> INTO <variable names>; Fetch is used to retrieve data from cursor.
- Close <cursor name>; Is used to close the cursor.

Following are the explicit cursor attributes available:
This is used to test whether a cursor is currently open so that this error can be avoided.

- **%found**

  Returns TRUE if operation has affected at least one row in the table else will return FALSE.
- **%notfound**

  Returns FALSE if operation has affected at least one row in the table else will return TRUE.

- **%rowcount**

  Returns number that no. of rows fetched from the cursor at any given time
- **%isopen**

  Returns TRUE if cursor is open else will return FALSE.
- **%bulk_rowcount**

  Returns rowcount for the bulk operation executed.
- **%bulk_exceptions**

  Returns exceptions raised for the bulk operation executed on a table (refer to FORALL statement)
  Declaring the Cursor:-

SQL>CURSOR <cursor_name> is <select_statement>;
Cursor_name- is a pl/sql identifier
Select_statement- is a SELECT statement with out an INTO clause.
Do not include the INTO clause in the cursor declaration.
If processing rows in a specific sequence is required, use the ORDER BY clause in the query.

Example:-

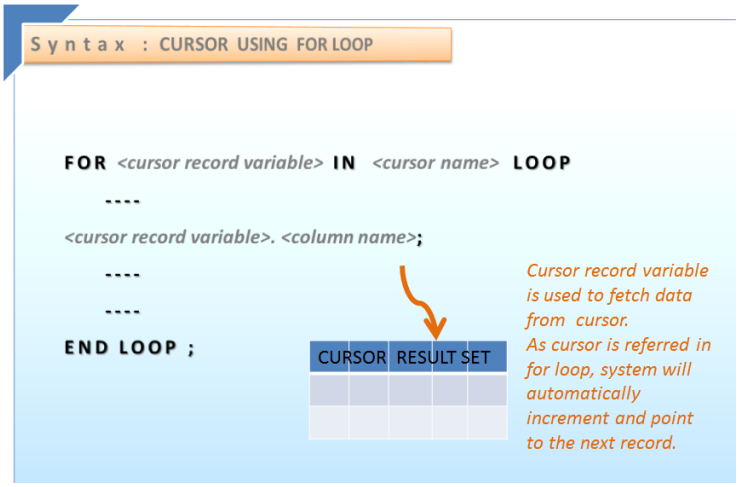
```
DECLARE
/*declaring the cursor based on the emp table*/
CURSOR cur_emp
IS
SELECT   empno, ename, sal, comm
FROM emp
ORDER BY deptno;
v_empno   emp.empno%TYPE;
v_ename   emp.ename%TYPE;
v_sal     emp.sal%TYPE;
v_comm    emp.comm%TYPE;
BEGIN
/* going to open the cursor*/
OPEN cur_emp;
/* if open then print the DBMS output*/
   IF cur_emp%ISOPEN
   THEN
   DBMS_OUTPUT.put_line ('Employee new Commission Report');
   END IF;
/* entering into loop and fetching the records one by one */
   LOOP
   FETCH cur_emp
   INTO v_empno, v_ename, v_sal, v_comm;
/* here we are going to exit the cursor if not found*/
   EXIT WHEN cur_emp%NOTFOUND;
   IF v_comm IS NULL
   THEN
   v_comm:= 255;
     ELSIF v_comm = 0
     THEN
     v_comm:= 300;
       ELSE
       v_comm:= v_comm + v_comm * .20;
   END IF;
   UPDATE emp
   SET comm = v_comm
   WHERE empno = v_empno;
   DBMS_OUTPUT.put_line (cur_emp%ROWCOUNT|| ' '|| v_empno|| ' '|| v_ename|| ' '|| v_sal|| ' '||
                                    v_comm);
   END LOOP;
  DBMS_OUTPUT.put_line('-----------------------------------------------------------------');
  DBMS_OUTPUT.put_line (cur_emp%ROWCOUNT|| ' '|| 'employees are updated with new commission');
 CLOSE cur_emp;
 COMMIT;
END;
```

O/P:

```
Employee new Commission Report  /* DBMS output */
1  7934  MILLER  1326  255
2  7839  KING  5100  255
3  7782  CLARK  2499  255
4  7844  TURNER  1560.6  300
5  7900  JAMES  988.38  255
6  7654  MARTIN  1300.5  1680
7  7499  ALLEN  1664.64  360
8  7698  BLAKE  2965.14  255
9  7521  WARD  1300.5  600
-----------------------------------------------------------------
9  employees are updated with new commission
```

Cursor using FOR Loop

- The cursor for loop is unique in that it does not require an explicit OPEN, FETCH, or CLOSE.
- The cursor FOR loop is a shortcut to process explicit cursors.
- Implicit open, fetch, exit, and close occur.
- The record is implicitly declared.
- Although the cursor is declared as an explicit cursor, PL/SQL handles its processing. In Addition, the for-loop uses a variable that is never declared in the DECLARATION section of the block.
- Using the same example as the other loops, we have rewritten the query to use the cursor for-loop instead.
- It is a shortcut because the cursor is opened, rows are fetched once for each iteration in the loop, the loop exists when the last row is processed, and the cursor is closed automatically.
- The loop itself is terminated automatically at the end of the iteration where the last row is fetched.



Record_name — is the name of the implicitly declare record.

Cursor_name — is a PL/SQL identifier for the previously declared cursor.

```
DECLARE
    /* declaring the cursor*/
CURSOR cur_emp
```

```
IS
SELECT * FROM emp;
BEGIN
  FOR v_ec IN cur_emp LOOP
   /*system automatically opens the cursor and fetch the data in a loop */
   DBMS_OUTPUT.PUT_LINE (v_ec.empno||'  '||v_ec.ename||'  '||v_ec.sal);
   END LOOP;
END;
```

OUTPUT:-

```
7839   KING   5100
7698   BLAKE   2965.14
7782   CLARK   2499
7499   ALLEN   1664.64
7521   WARD   1300.5
7654   MARTIN   1300.5
7844   TURNER   1560.6
7900   JAMES   988.38
7934   MILLER   1326
```

'Cursor – for Loop' Using Sub Queries:-

- When you use a nested block in a FOR loop, you do not need to declare a cursor.

```
DECLARE
     /* declaring the cursor*/
CURSOR cur_emp
IS
SELECT * FROM emp;
BEGIN
  FOR v_ec IN cur_emp LOOP
   /*system automatically opens the cursor and fetch the data in a loop */
   /* Declaring a nested block in FOR loop */
    BEGIN
    DBMS_OUTPUT.PUT_LINE (v_ec.empno||'  '||v_ec.ename||'  '||v_ec.sal);
    END;  /* End of nested block in FOR loop */
   END LOOP;
END;
DBMS_OUTPUT.PUT_LINE (v_ec.empno||'  '||v_ec.ename||'  '||v_ec.sal);
END LOOP;
END;
```

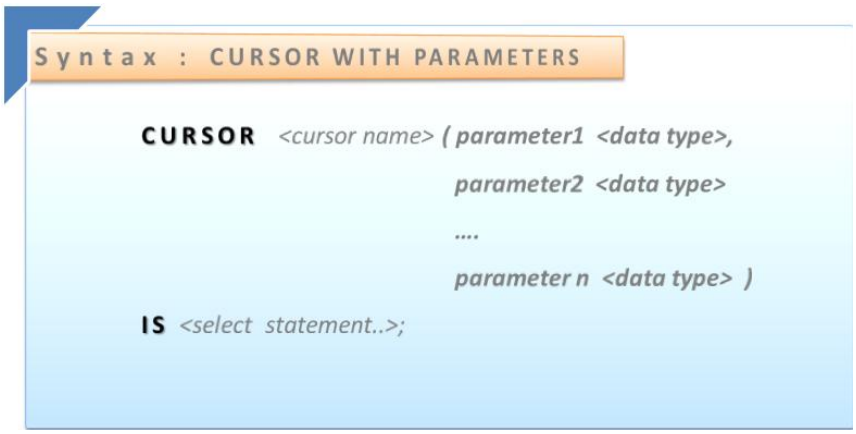
Output:-

```
7839   KING   5100
7698   BLAKE   2965.14
7782   CLARK   2499
7499   ALLEN   1664.64
7521   WARD   1300.5
7654   MARTIN   1300.5
```

7844 TURNER 1560.6
7900 JAMES 988.38
7934 MILLER 1326

Parameterized Cursors

- A **parameterized cursor** is another way of handling cursor with parameter just like pl/sql stored procedures or functions.
- Using parameterized cursor it is possible to open the same cursor multiple times with different parameters values.
- The main advantage of using parameterized cursor is to flexibility and avoidance of the scoping problem i.e. like a cursor can be defined in any block and can be used in any other block.
- This means that you can open and close an explicit cursor several times in a block, returning a different active set on each occasion.
- For each execution, the previous cursor is closed and re-opened with a new set of operators.



- Cursor_name — is a PL/SQL identifier for the previously declared cursor.
- Parameter_name — is the name of the parameter.

Open the cursor with parameter:-
Syntax:-

```
OPEN <cursor_name> (parameter_name/value…);
For <record_name> in cursor_name (parameter_name/value)    /*It is for FOR Loop*/
```

Example 1:

```
DECLARE
  CURSOR emp_cur (v_deptno in number)
  IS   /* Here declaring the cursor with parameter i.e. in type */
  SELECT * FROM emp
  WHERE deptno=v_deptno;
BEGIN
   FOR v_ec IN emp_cur(30) LOOP    /* Here we are passing the parameter value-30*/
   DBMS_OUTPUT.PUT_LINE (v_ec.empno||'  '||v_ec.ename||'  '||v_ec.sal);
   END LOOP;
END;
```

Output:-

```
7698   BLAKE   2965.14
7499   ALLEN   1664.64
7521   WARD   1300.5
7654   MARTIN   1300.5
7844   TURNER   1560.6
7900   JAMES   988.38
```

Example 2:-
In this example we can explain parameterized cursor by using OPEN, FETCH and CLOSE syntax.

```
DECLARE
  CURSOR emp_cur (v_deptno IN NUMBER) /* Here declaring the cursor with parameter i.e. in type */
  IS
  SELECT * FROM emp
  WHERE deptno=v_deptno;
  v_ec emp_cur%ROWTYPE;   /* Declaring the variable with same as cursor type */
BEGIN
  OPEN emp_cur(10);
   LOOP /* Here we are passing the parameter value ->10*/
     FETCH emp_cur INTO v_ec; /* Fetch the records from active set (emp_cur) to variable */
     EXIT WHEN emp_cur%NOTFOUND; /* terminate the loop by using cursor variable */
     DBMS_OUTPUT.PUT_LINE (v_ec.empno||'  '||v_ec.ename||'  '||v_ec.sal);
   END LOOP;
  CLOSE emp_cur; /* close the cursor */
END;
```

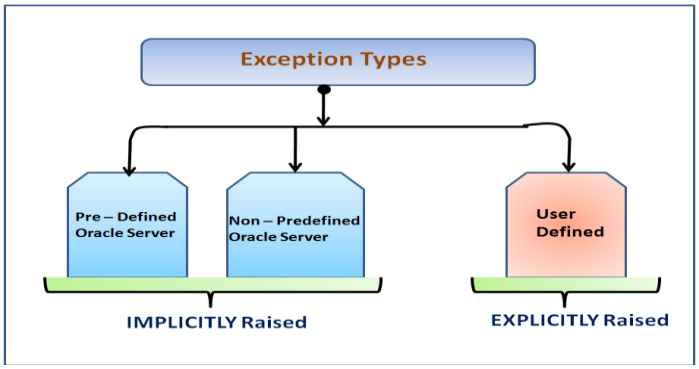
Output:-

```
7839   KING   5100
7782   CLARK   2499
7934   MILLER   1326
```

Chapter 7: Error Handling

What is an EXCEPTION?

- **Exception** is used to handle the errors in pl/sql block.
- Error handling solutions are provided in exception block.
- PL/SQL allows developers to raise and handle errors/exceptions in a very flexible way using exception handlers.
- Exception can be associated with an oracle system error or with a user-defined error.
- By using exceptions and exception handlers, we can make your PL/SQL programs robust and able to deal with both unexpected and expected errors during execution of the program.
- When exception is raised, control passes to the exception section of the block. The exception section consists of handlers for some or all of the exceptions. An exception handler contains the code that is executed when the error associated with the exception occurs, and the exception is raised.

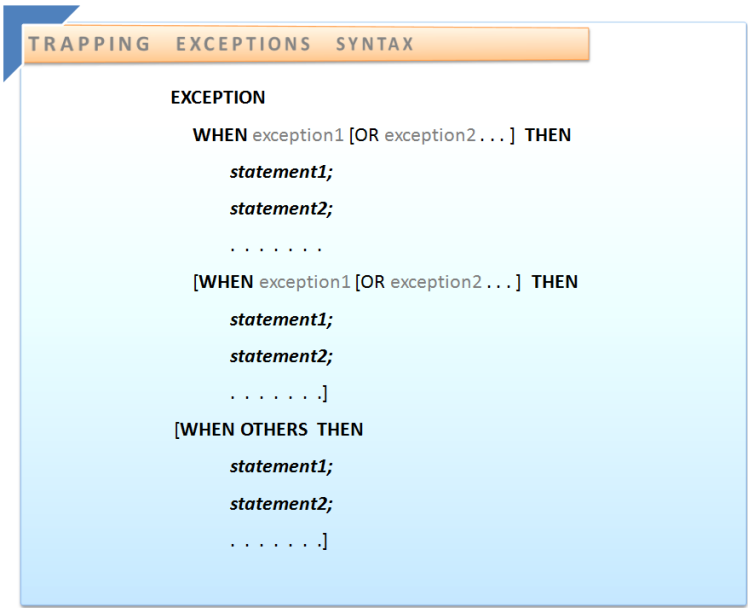


Exception Types

You can program for exceptions to avoid disruption at run time. There are three types of exceptions.

Exception	Description	Directions for Handling
Predefined Oracle Server error	One of approximately 20 errors that occur most often in PL/SQL code	Do not declare and allow the Oracle server to raise them implicitly
Nonpredefined Oracle Server error	Any other standard Oracle Server error	Declare within the declarative section and allow the Oracle Server to raise them implicitly
User-defined error	A condition that the developer determines is abnormal	Declare within the declarative section, *and* raise explicitly

Note: Some application tools with client-side PL/SQL, such as Oracle Developer Forms, have their own exceptions.



Trapping Exceptions:
We can trap any error by including a corresponding routine within the exception handling section of the PL/SQL block. Each Handler consists of WHEN clause, which specifies an exception, followed by a sequence of statements to be executed when that exception is raised.

Exception	is the standard name of a predefined exception or the name of the user defined

	exception declared within the declarative section.
Statement	is one or more PL/SQL statements
Others	is an optional exception handling clause that traps unspecified exceptions

Trapping Exceptions guidelines:

- The EXCEPTION keyword starts exception-handling section
- Several exception handlers are allowed
- Only one handler is processed before leaving the block
- WHEN OTHERS is the last clause

Pre-Defined Exceptions

- These are defined by oracle
- Activated by oracle automatically
- Solution provided by user
- An internal exception is raised implicitly when ever our pl/sql program violates on oracle rule or exceeds a system-dependent limit.
- Every oracle error has number, but exceptions must be handled by name.
- So, PL/SQL predefines some common oracle errors as exceptions.
- For example, PL/SQL raises the predefined exception NO_DATA_FOUND if a SELECT INTO statement returns no rows.
- To handle other oracle errors, you can use the OTHERS handler. The error-reporting functions SQLCODE and SQLERRM are especially useful in OTHERS handler because they return the Oracle error numbers.
- PL/SQL declares predefined exceptions globally in package STANDARD, which defines the PL/SQL environment. So, you need not declare them our self.

We can write handlers for predefined exceptions using the names shown below.

NO_DATA_FOUND (ora-01403/100):- Automatically activated whenever select statement retrieves more than one row into variables.

TOO_MANY_ROWS (ORA-01422):-Automatically activated whenever select statement retrieves more than one row into variables

VALUE_ERROR (ORA-06502):- Automatically activated whenever datatype/size of variables are not matching.

ZERO_DIVIDE (ORA-01746):- Automatically activated whenever invalid arithmetic operation is performed.

DUP_VAL_ON_INDEX (ORA-000001):- Automatically activated whenever duplicate values are inserted into unique-index column.

STORAGE_ERROR (ORA-06500):- Automatically activated whenever there is lack of memory in server.

INVALID_CURSOR (ORA-01001):- Automatically activated whenever user fetch the rows from cursor with out opening the cursor.

CURSOR_ALREADY_OPEN (ORA-06511):- Automatically activated whenever user opens the cursor

which is already in open state.

TIMEOUT_ON_RESOURCES (ORA-00051):- Automatically activated whenever user performs an infinite loop process

TRANSACTION_BACKED_OUT:- Automatically activated whenever there is an communication problem with server.

LOGIN_DENIED (ORA-01017):- Automatically activated whenever user name or password are not valid.

OTHERS: - General Exception
Used to handle any predefined exception . It will not support user defined exceptions.

Pseudo columns activated by exception .
1. SQLCODE: – holds currently raised error number.
2. SQLERRM:- holds currently raised error message
It must be last exception in exception block

Example1:-

Here we are going to know the where and how to use NO_DATA_FOUND Exception

```
DECLARE
  v_empno  NUMBER (10);
  v_ename  VARCHAR2 (10);
  v_sal  NUMBER (10);
  v_comm  NUMBER (10);
  v_job VARCHAR2 (10);
BEGIN
v_ename  := 'BLAKE';
SELECT ename, sal,   comm,    job
  INTO v_ename, v_sal, v_comm,  v_job FROM  emp
  WHERE ename = v_ename ;
  DBMS_OUTPUT.put_line (   'empno'|| '  '|| 'ename'|| '  '|| 'sal'|| '  '|| 'comm'|| '  '|| 'job');
  DBMS_OUTPUT.put_line('-----------------------------------------------------------------------------');
  DBMS_OUTPUT.put_line (   v_empno|| '  '|| v_ename|| '  '|| v_sal|| '  '|| v_comm|| '  '|| v_job);
EXCEPTION
  WHEN NO_DATA_FOUND
  THEN
    DBMS_OUTPUT.put_line ('No such employee exists with name -->' || v_ename);
END;
```

Output:-

```
empno  ename  sal  comm  job
-----------------------------------------------------------------------------
  BLAKE  2965  255  MANAGER
```

/* Now let us give another input value for a variable **v_ename** i.e., after begin
give the variable value as : "**v_ename** := 'PAVAN';"
early it was **v_ename** := 'BLAKE'; */

-- now see the following output you can see it if you run the above program
Output:-
anonymous block completed /* Script output */
No such employee exists with name -->PAVAN /* DBMS output */

Example2:-
Here we are going to know how and where to use TOO_MANY_ROWS EXCEPTION

```
DECLARE
  v_dept  NUMBER (10);
  v_ename  VARCHAR2 (10);
BEGIN
  v_dept := 10;
/* here we are going to select 'ename' based on 'deptno' which returns more records , but select statement
fetch only one into variable*/
  SELECT ename
  INTO v_ename
  FROM emp
  WHERE deptno = v_dept;
EXCEPTION
   WHEN TOO_MANY_ROWS
  THEN
    DBMS_OUTPUT.put_line ('There are too many rows in this deptno-->'|| v_dept);
END;
```

Output:-
There are too many rows in this deptno-->10 /* DBMS output */

SQLCODE and SQLERRM:

SQLCODE and SQLERRM are standard keywords to fetch the details of the exception.

SQLCODE: gives the error code.
SQLERRM: gives the error message.

```
BEGIN
EXCEPTION
WHEN OTHERS THEN
 dbms_output.put_line ('Exception raised'||sqlcode||sqlerrm);
 END;
```

Example3:-

Here we are going to know usage of OTHERS

```
DECLARE
  v_empno  NUMBER (10);
  v_ename  VARCHAR2 (10);
  v_sal  NUMBER (10);
  v_comm  NUMBER (10);
  v_job  VARCHAR2 (10);
BEGIN
  v_empno := 55;
/*fetching the record of the employee based on empno.
If empno is there then it will display the details of the corresponding employee details
otherwise it goes to OTHERS exception*/
  SELECT ename, sal,   comm,    job
```

```
  INTO v_ename, v_sal, v_comm,  v_job
  FROM  emp
  WHERE empno = v_empno;
  DBMS_OUTPUT.put_line (   'empno'|| '  '|| 'ename'|| '  '|| 'sal'|| '  '|| 'comm'|| '  '|| 'job');
  DBMS_OUTPUT.put_line (   v_empno|| '  '|| v_ename|| '  '|| v_sal|| '  '|| v_comm|| '  '|| v_job);
EXCEPTION
  WHEN OTHERS THEN
  THEN
    DBMS_OUTPUT.put_line (SQLCODE||'-------  '||SQLERRM);
END;
```

Output:-

100------- ORA-01403: no data found

Unnamed system exceptions

- These are defined by user.
- Activated automatically by constraints.
- Solution provided by user.
- Used to handle constraint violation error in PL/SQL block.
- These can be handled by using

PRAGMA EXCEPTION_INIT.

Some of the system exceptions with error codes between the range -20999 through -20000 don't have a exception name. So for these unnamed system exceptions we can define a name using PRAGMA EXCEPTION_INIT and handle it accordingly.

These exceptions do not occur frequently as a reason Oracle has not provided the names. EXCEPTION_INIT will associate a named exception with a particular oracle error. This gives you the ability to trap this error specifically, rather than via an OTHERS handler.

Syntax:

?

```
1
    PRAGMA EXCEPTION_INIT (exception_name, oracle_error_number);
```

Exception_name - *is the name of an exception declared prior to the PRAGMA*
Oracle_error_number - *is the desired error code to be associated with this named exception.*
This PRAGMA must be in declarative section.

PRAGMA EXCEPTION_INIT:-

PRAGMA: – It is a compiler directive that is processed at compile time not at runtime

EXCEPTION_INIT:- built in sub-program used to provide exception name to the constraint violation error number.

Example:-

Here we are going to know the non-predefined exception and pragma exception init

```
DECLARE
/*here we are declaring the exception which we are going to handle in exception block*/
  deptno_exists   EXCEPTION;
/*PRAGMA EXCEPTION_INIT which handles non-predefined exception and which should declare in declarative
section with exception_name and corresponding constraint error number*/
  PRAGMA EXCEPTION_INIT (deptno_exists, -00001);
BEGIN
  INSERT INTO dept
     VALUES (10, 'TESTING', 'FLORIDA');
```

```
EXCEPTION
  WHEN deptno_exists
  THEN
    DBMS_OUTPUT.put_line ('DEPTNO ALREADY EXISTS WITH THIS');
END;
```

O/P:
DEPTNO ALREADY EXISTS WITH THIS
PL/SQL procedure successfully completed.

User Defined Exceptions

- These are defined by user.
- Activated by user using "RAISE" statement.
- Solution provided by user
- User defined exceptions are declared in the declarative section of a PL/SQL block. Just like variables , exceptions have a type(EXCEPTION) and a scope

Example:-
/*Here we are going to know how to handle User Defined Exception*/

```
DECLARE
  v_empno  emp.empno%TYPE ;
  v_ename  emp.ename%TYPE:='PAVAN';
  v_sal  emp.sal%TYPE;
  v_comm  emp.comm%TYPE;
  sal_missing  EXCEPTION;   /* user defined exception. We are going to use it in Exception block*/
BEGIN
/* here we are going to fetch records into select statement*/
  SELECT ename, sal, comm
  INTO v_ename, v_sal, v_comm
  FROM emp
  WHERE ename=v_ename ;
    UPDATE emp
    SET sal = v_salWHERE ename = v_ename;
  IF v_sal IS NULL
  THEN
  RAISE sal_missing;
  end if;
    DBMS_OUTPUT.put_line (v_empno|| '  '|| v_ename|| '  '|| v_sal|| ' '|| v_comm);
EXCEPTION
  WHEN NO_DATA_FOUND
    THEN
    DBMS_OUTPUT.put_line ('No such employee exists with-->' || v_ename );
  WHEN TOO_MANY_ROWS
    THEN
    DBMS_OUTPUT.put_line ('too many rows are retrieving');
  WHEN sal_missing
    THEN
    DBMS_OUTPUT.put_line ('salary is null-providing basic pay');
      UPDATE emp
      SET sal = 1255
      WHERE ename = v_ename;
  WHEN VALUE_ERROR
    THEN
```

```
    DBMS_OUTPUT.put_line ('value error');
  WHEN OTHERS
    THEN
    DBMS_OUTPUT.put_line (SQLCODE || ' ' || SQLERRM);
END;
```

O/P:

* If you want to see the user defined exception sal_missing then before executing the above code insert the following data into 'emp' table.

SQL> INSERT INTO emp VALUES(1010,'PAVAN','MANAGER',7839,'05-SEP-2004',NULL,500,20);
now if you observe we are passing 'null' value to SALARY column of emp table, so therefore according to the above program logic our sal_missing will execute. And the output is :-
anonymous block completed /* script output */
salary is null-providing basic pay /* DBMS output */

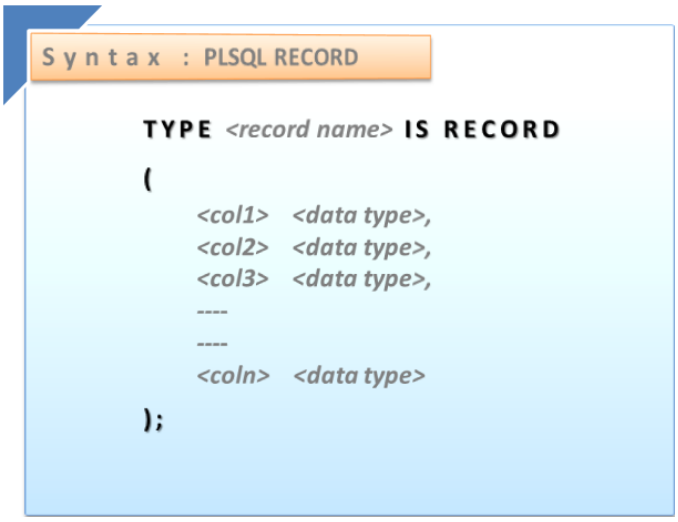
Chapter 8: Record Types

Record Type In PL/SQL

Records are called composite data types, as record is a combination of multiple scalar datatypes like char, varchar2, number etc.
Each field in the Pl/SQL record has its own name and data type. Records are always created in the declaration section.

A record can be created in two ways:

USING GENERAL SYNTAX:
A record can be visualized as a row of data.



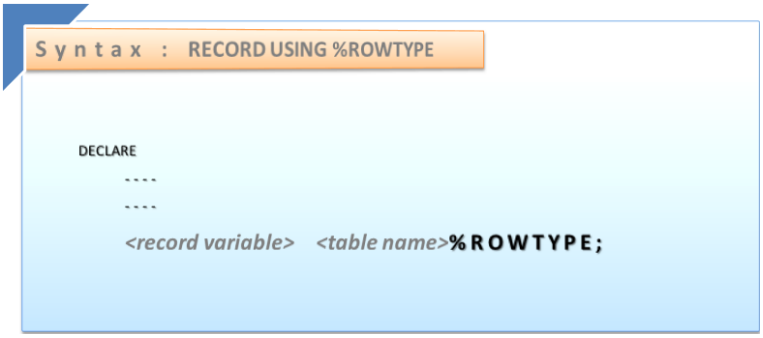

```
TYPE  Emp_type IS RECORD
( empno     NUMBER(5),
  ename     VARCHAR2(25),
  job       VARCHAR2 (25),
  sal       NUMBER(10)
```

```
 hire_date DATE );
Emp_rec    Emp_type;
```

From the above statement, Emp_rec is created with type Emp_type;

USING %ROWTYPE:

%ROWTYPE is used to refer to table structure. When we use %ROWTYPE system will internally create a record with the structure same as table record.



Example:-

```
Emp_rec    EMP%rowtype;
```

From the above statement 'Emp_rec' record will automatically inherit the record structure of emp table. For every column in the table, there's will be a field in the record with the same name and data type as the column i.e. used to define the variable according to complete table structure.

Referencing Fields of Records

Individual record fields are referenced via dot notation:

<record_name>.<field_name>

Record Assignment
Values can be assigned to records or to the fields within a record in four different ways:

- The assignment operator can be used to assign a value to a field:

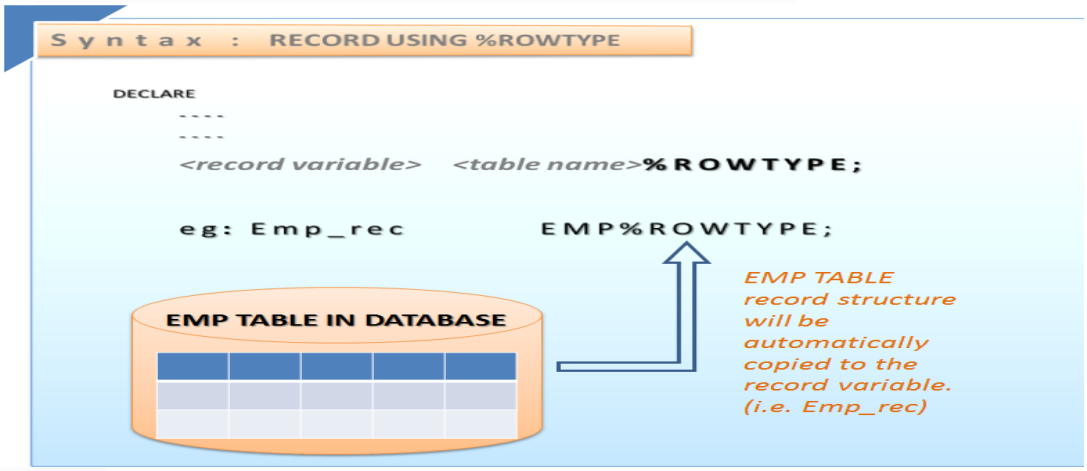
```
Emp_rec.hire_date := SYSDATE;
```

- We can SELECT INTO a whole record or the individual fields:

```
SELECT empno,ename,job,sal,hire_date
  INTO Emp_rec
  FROM emp
 WHERE ename = 'FORD';
```

Record Using %ROWTYPE

- Record created with %ROWTYPE will inherit the structure of the table.

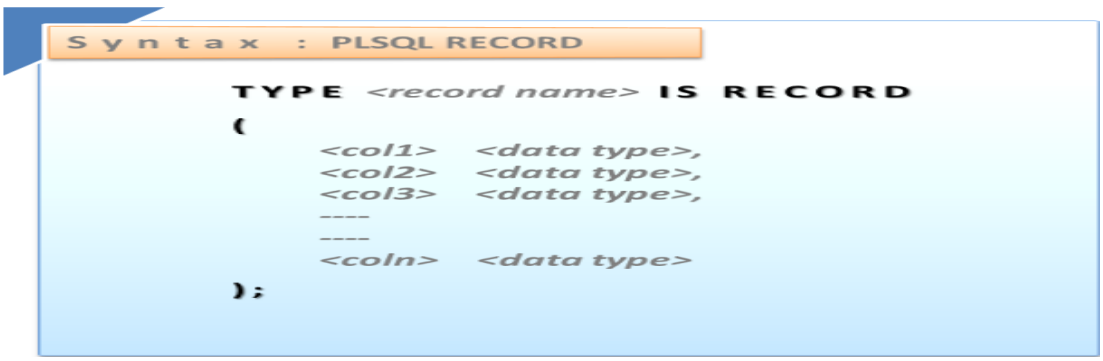


Example:-
Using %ROWTYPE declaration

```
SQL> DECLARE
  2  /*declaring a variable as 'emp_rec', now if we want to use any column from the table we should
use dot operator along with 'emp_rec' shown below*/
  3     emp_rec  emp%ROWTYPE;
  4  BEGIN
  5     emp_rec.ename := 'BLAKE';  /*fetching the records into 'emp_rec'*/
  6     SELECT * INTO emp_rec FROM emp
  7     WHERE ename=emp_rec.ename;
  8    DBMS_OUTPUT.put_line( emp_rec.empno ||' '|| emp_rec.ename ||' '|| emp_rec.sal||' '||emp_rec.
job||' '||emp_rec. hiredate||' '||emp_rec.comm||' '||emp_rec. deptno);
  9  END;
 10  /
7698 BLAKE 3024.44 MANAGER 01-MAY-81 600 30
PL/SQL procedure successfully completed.
```

Record Using General Syntax

- Using general syntax we first create record type with some structure. A record can be visualized as a row of data.
- Once we create the record type we need to create record variable using the type.



```
SQL> DECLARE
  2  /* creating record type 'Emp_rec' for all the types in 'emp' table  */
  3    TYPE  Emp_type IS RECORD
  4        (empno     NUMBER(5),
  5         ename     VARCHAR2(25),
  6         job       VARCHAR2 (25),
  7         mgr       NUMBER(5),
  8         hire_date DATE,
  9         sal       NUMBER(10),
 10         comm      NUMBER,
 11         deptno    NUMBER
 12         );
 13    emp_rec   Emp_type;  /* Declaring a record variable 'emp_rec' with record type 'emp_type' */

 14  BEGIN
 15  /* now if we want to use any column from the table we should use dot operator along with 'Emp_r
ec' as shown below */
 16    emp_rec.ename := 'KING';
 17  /* fetching the records into 'Emp_rec' */
 18    SELECT * INTO emp_rec
 19    FROM emp
 20    WHERE ename = emp_rec.ename;
 21  /* Printing values that are fetched into 'Emp_rec' */
 22    DBMS_OUTPUT.put_line( emp_rec.empno|| '  '||emp_rec.ename|| '  '|| emp_rec.sal||'  '|| emp_r
ec.job );
 23  END;
 24  /
7839  KING  5202  PRESIDENT
PL/SQL procedure successfully completed.
```

chapter:9-sub programs

Stored Procedure

- A **Procedure** is a named PL/SQL block that can accept parameters (some times referred to as arguments), and be invoked.
- Generally speaking we use a procedure to perform an action.
- A procedure has a header, a declaration section, an executable section and an optional exception-handling section.
- A procedure can be compiled and stored in database as a schema object.
- Procedures promote reusability and maintainability. When validated, they can be used in any number of applications.
- If the requirements change, only the procedure needs to be updated.
- Using “CREATE OR REPLACE” command we can replace the procedure that already exists in the database with the same name.
- A procedure may or may not return any value.

Syntax : PROCEDURE

```
CREATE [OR REPLACE] PROCEDURE <procedure name> ( parameter1  <data type>,
                                                 parameter2  <data type>
                                                 ....
                                                 parameter n  <data type> )
IS|AS
      <declaration section>
BEGIN
      <execution section>
EXCEPTION
      <exception section>

END  <procedure name> ;
```

Syntax Definition:-

Procedure_name	Name of the procedure.
Parameter	Name of a PL/SQL variable whose value is passed to or populated by theCalling environment.
Mode	Type of argument IN (default), OUT, INOUT.

```
SQL> DECLARE
  2  /* creating record type 'Emp_rec' for all the types in 'emp' table  */
  3    TYPE  Emp_type IS RECORD
  4        (empno     NUMBER(5),
  5         ename     VARCHAR2(25),
  6         job       VARCHAR2 (25),
  7         mgr       NUMBER(5),
  8         hire_date DATE,
  9         sal       NUMBER(10),
 10         comm      NUMBER,
 11         deptno    NUMBER
 12         );
 13    emp_rec   Emp_type;  /* Declaring a record variable 'emp_rec' with record type 'emp_type' */

 14  BEGIN
 15  /* now if we want to use any column from the table we should use dot operator along with 'Emp_r
ec' as shown below */
 16    emp_rec.ename := 'KING';
 17  /* fetching the records into 'Emp_rec' */
 18    SELECT * INTO emp_rec
 19    FROM emp
 20    WHERE ename = emp_rec.ename;
 21  /* Printing values that are fetched into 'Emp_rec' */
 22    DBMS_OUTPUT.put_line( emp_rec.empno|| ' '||emp_rec.ename|| ' '|| emp_rec.sal||' '|| emp_r
ec.job );
 23  END;
 24  /
7839  KING  5202  PRESIDENT
```

PL/SQL procedure successfully completed.

How to execute a procedure:-

By using “exec” or “execute” command to run the procedure in SQL * Plus. shortcut keys are allowed i.e. “exec” .

```
SQL> /* CREATE OR REPLACE will overwrite any existing object with the same name*/
SQL> CREATE OR REPLACE PROCEDURE emp_info_proc
  2  IS
  3  /* Declare a cursor based on “emp” table */
  4    CURSOR emp_cur
  5    IS
  6      SELECT empno, sal, deptno
  7      FROM emp;
  8  BEGIN
  9    DBMS_OUTPUT.PUT_LINE ('EMPNO ENAME   SAL');
 10    DBMS_OUTPUT.PUT_LINE ('------------------------------');
 11  /*below For Loop will automatically open and fetch the records into “emp_rec”*/
 12    FOR emp_rec IN emp_cur   LOOP
 13      DBMS_OUTPUT.PUT_LINE (emp_rec.empno|| '  '|| emp_rec.sal|| '  '|| emp_rec.sal);
 14    END LOOP;
 15  END emp_info_proc;  /*End of the Procedure*/
 16  /

Procedure created.

SQL>
SQL> EXECUTE EMP_INFO_PROC;
EMPNO ENAME   SAL
------------------------------
7499   1697.93   1697.93
7521   1326.51   1326.51
7654   1326.51   1326.51
7698   3024.44   3024.44
7782   2548.98   2548.98
7839   5202   5202
7844   1591.81   1591.81
7900   1008.15   1008.15
7934   1352.52   1352.52
856   4745.26   4745.26
23   4745.26   4745.26

PL/SQL procedure successfully completed.
```

To see the Procedure availability:-

SELECT *object_name,object_type,status*
FROM user_objects
WHERE *object_name* = <Procedure Name>

To see the procedure source code:-

SQL> **SELECT** *text* FROM user_source **WHERE** *name* =<procedure_name>;
Example:-

SELECT text FROM user_source WHERE name='EMP_INFO_PROC';

```
SQL>   SELECT  text FROM user_source WHERE name ='EMP_INFO_PROC';
TEXT
--------------------------------------------------------------------------------
PROCEDURE emp_info_proc
IS
/* Declare a cursor based on “emp” table */
  CURSOR emp_cur
  IS
    SELECT empno, sal, deptno
    FROM emp;
BEGIN
  DBMS_OUTPUT.PUT_LINE ('EMPNO ENAME   SAL');
  DBMS_OUTPUT.PUT_LINE ('------------------------------');
/*below For Loop will automatically open and fetch the records into “emp_rec”*/
  FOR emp_rec IN emp_cur   LOOP
    DBMS_OUTPUT.PUT_LINE (emp_rec.empno|| '   '|| emp_rec.sal|| '   '|| emp_rec.sal);
  END LOOP;
END emp_info_proc;  /*End of the Procedure*/

15 rows selected.

SQL> EXECUTE EMP_INFO_PROC;
EMPNO ENAME   SAL
------------------------------
7499   1697.93   1697.93
7521   1326.51   1326.51
7654   1326.51   1326.51
7698   3024.44   3024.44
7782   2548.98   2548.98
7839   5202   5202
7844   1591.81   1591.81
7900   1008.15   1008.15
7934   1352.52   1352.52
856   4745.26   4745.26
23   4745.26   4745.26

PL/SQL procedure successfully completed.
```

Recompile an existing procedure:-

ALTER PROCEDURE **<procedure_name>** COMPILE**;**

Dropping the procedure from the Database:-

DROP PROCEDURE **<procedure_name>;**
Example:-

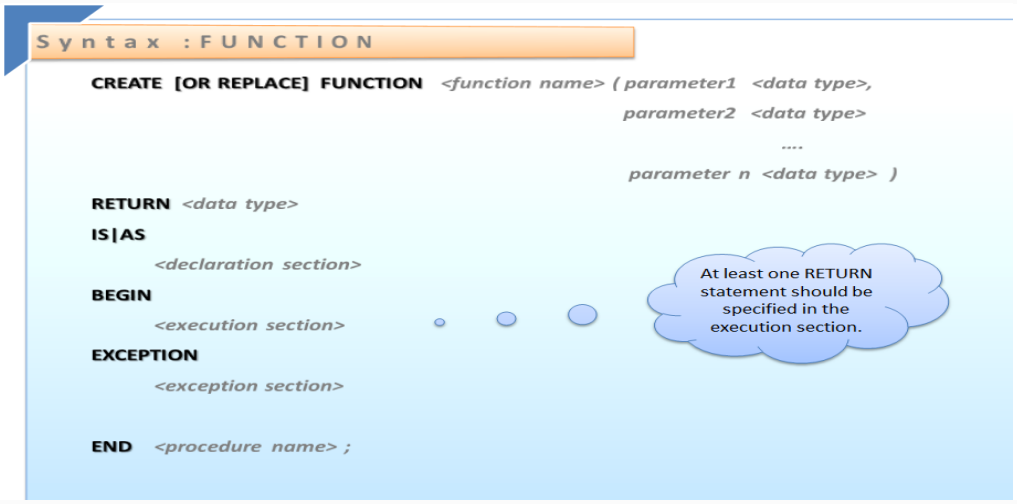
DROP PROCEDURE EMP_INFO_PROC;

Output:-

procedure EMP_INFO_PROC dropped. /* Script Output */

PL/SQL Function

- A **Function** is a named PL/SQL block that returns a value. A Function is a subprogram that computes a value.
- A function must return a value to the calling environment, whereas a procedure returns zero or more values to its calling environment.
- Function can be stored in the database as a schema object for repeated execution.
- A function stored in the database is referred to as a stored function. A function must have the RETURN clause in the header and at least one RETURN statement in the executable section.
- Function promotes reusability and maintainability. When validated they can be used in any number of applications.
- Function is called as part of a SQL expression or as part of a PL/SQL expression.



Syntax Definition:-

Function_name	Name of the function.
Parameter	Name of a PL/SQL variable whose value is passed to or populated by theCalling environment.
Mode	Type of argument IN (default), OUT, INOUT.
RETURN	Datatype of the return value that must be output by the function

Example1:-function without parameter:

```
SQL> CREATE OR REPLACE FUNCTION emp_deptno_count
 2  RETURN NUMBER    /*We try to create the function with RETURN clause and data type is NUMBER*/
 3  IS
 4    v_count   NUMBER;
 5  BEGIN
 6      SELECT COUNT (*)
 7      INTO v_count
 8      FROM emp
 9      WHERE deptno = 10;
10    RETURN (v_count);      /* Returning the value */
11  EXCEPTION
12    WHEN OTHERS           /* Exception Block */
13    THEN
```

```
 14     RETURN 0;
 15  END emp_deptno_count;
 16  /
```

Function created.

Execute a Function:-
By using “select” statement we execute the functions.

```
SQL> SELECT emp_deptno_count FROM dual;
EMP_DEPTNO_COUNT
----------------
               5
```

Function with parameter:

```
SQL> CREATE OR REPLACE FUNCTION emp_avg_sal_func (p_deptno emp.deptno%TYPE)
  2     RETURN NUMBER
  3     IS
  4       v_avg_sal   NUMBER;
  5  BEGIN
  6     /* Here we try to find out the average salary based on deptno*/
  7     SELECT TRUNC (AVG (sal), 3)
  8     INTO v_avg_sal
  9     FROM emp
 10      WHERE deptno = p_deptno;
 11     RETURN (v_avg_sal);  /*Returning the value*/
 12  EXCEPTION
 13     WHEN OTHERS
 14     THEN
 15       RETURN 0;
 16  END emp_avg_sal_func;
 17  /
```

Function created.

To see the output of the function giving parameter values look at the following example code:-

```
SQL> SELECT emp_avg_sal_func (10) FROM DUAL;
EMP_AVG_SAL_FUNC(10)
--------------------
          3718.804
```

Or we can use this way.

```
SQL> DECLARE  /* First declare a variable */
  2     RESULT   NUMBER;
  3  BEGIN
  4         /*Here call the function and assign the value to the variable */
  5     RESULT := emp_avg_sal_func (10);
  6     DBMS_OUTPUT.put_line ('Average Sal->' || RESULT);
  7  END;
  8   /
Average Sal->3718.804
PL/SQL procedure successfully completed.
```

To see the Function availability:-

To see the Function source code:-

SELECT *text* **FROM** *user_source* **WHERE** *name* =<function_name>;

Parameters In Procedures And Functions

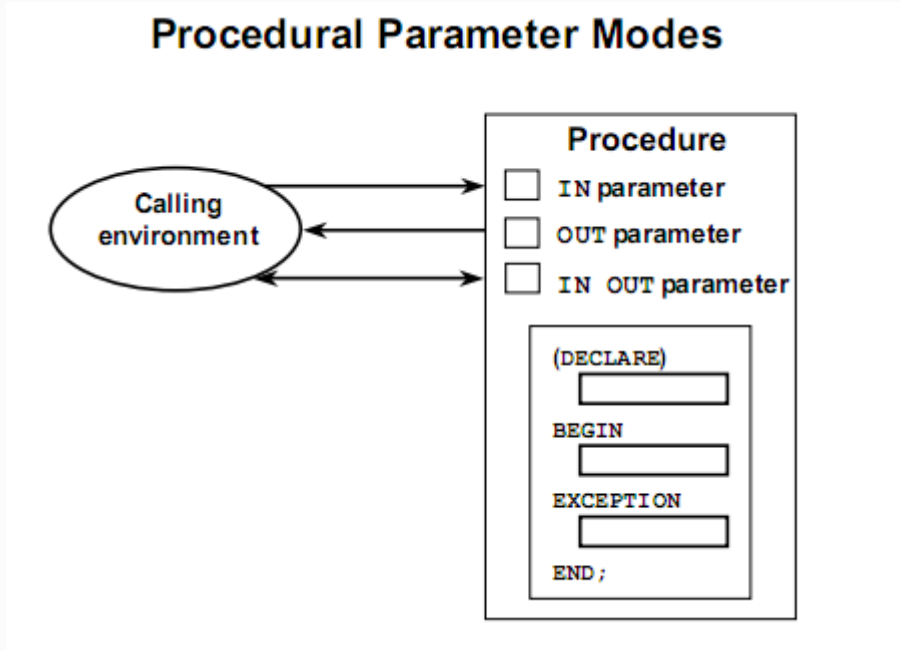
Formal Parameters:-

- Variables declared in the parameter list of subprogram specification. Parameters In Procedures And Functions

Example:-

CREATE PROCEDURE raise_sal (p_id NUMBER, p_amount NUMBER)
…….
…….
…….
…….
END raise_sal

Parameter modes in Procedures and Functions:-



Type of Parameter	Description
IN	Passes a constant value from the calling environment into the procedure..
OUT	Passes a value from the procedure to the calling environment.
IN OUT	Passes a constant value from the calling environment into the procedure and a possibly different value from the procedure back to the calling environment Using the same parameter.

IN Parameter:-

- Passes a value into the program, from the calling environment.
- It is exactly same as a constant i.e. initial value is must and that value remains unchanged throughout the execution of the program.
- Best suitable for constants, literals, expressions.

- IN is the default type of parameter.
- Procedure/functions can have any number of IN parameters and IN parameters can have default values.

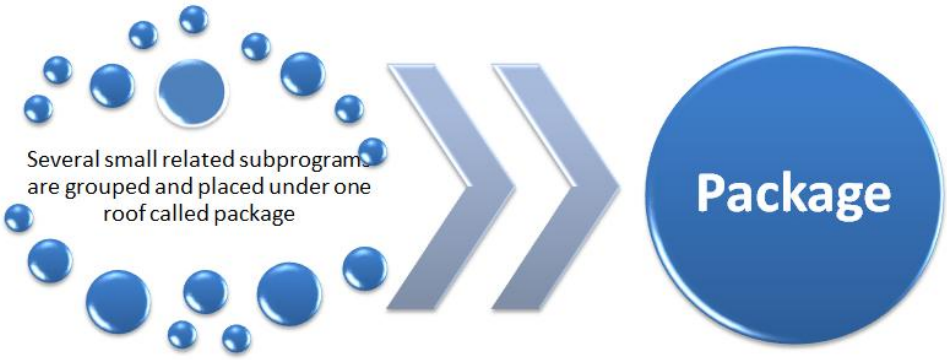
PL/SQL Tutorial - Subprogram Dependences

- A stored subprogram is marked as invalid in the data dictionary if it has compile errors.
- A stored subprogram can also become invalid if a DDL operation is performed on one of its dependent objects.
- If a subprogram is invalidated, the PL/SQL engine will automatically attempt to recompile in the next time it is called.
- If we have two procedures like P1 and P2 in which P1 depends on P2. If we compile P2 then P1 is invalidated.

Chapter 10: Packages

What is a Package?

- A package is PL/SQL construct that allows related objects to be stored together .
- A package is a container for related objects.
- Package is a method to handle all the required functions and procedures .



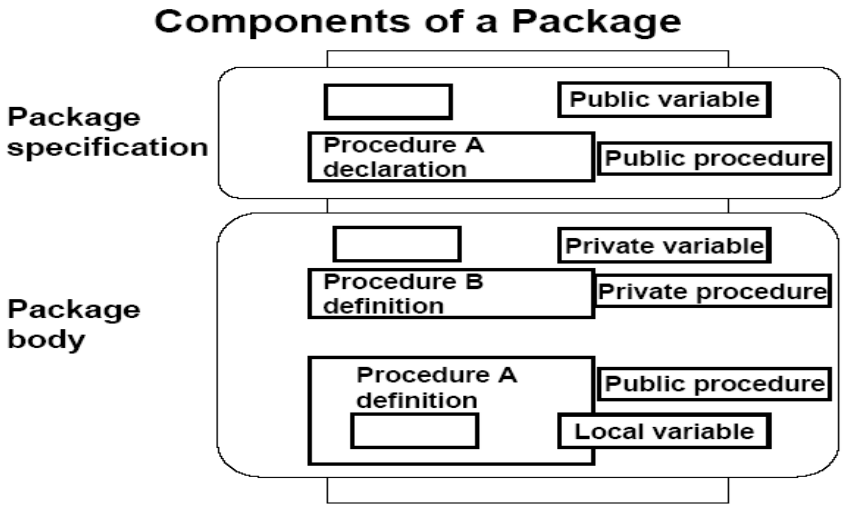
- All of the code is loaded on the first call of the package. The first call to the package is very expensive but all subsequent calls will result in an improved performance.
- Packages should be taken very seriously in all such applications where procedures and functions are used repeatedly.
- The structure of a PACKAGE encourages the TOP-DOWN approach.
- Similar to standalone procedures and functions, packaged procedures and functions can be called explicitly by applications or users.
- Organized routines.
- Increased functionality (e.g. global package variables can be declared and used by any procedure in the package) .
- Increased performance (e.g. all objects of the package are parsed, compiled, and loaded into memory once).

- The first time a packaged subprogram is called then the package is instantiated.
- Each session will have its own copy of packaged variables, ensuring that two sessions executing subprograms of the same package use different memory locations.

Components Of A Package

A well designed package is a logical grouping of objects such as

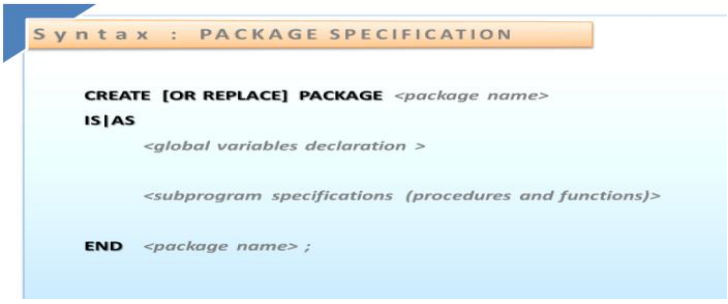
- Functions
- Procedures
- Global Variables
- Cursors.



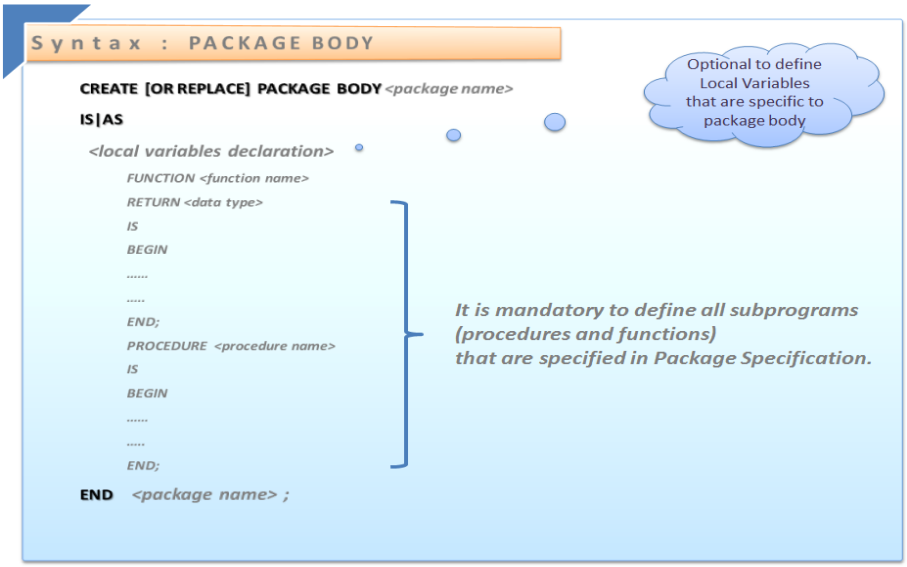
Package Contains two parts :

- Package Specification
- Package Body

Package Specification.



Package Body:



Example:-Package Specification

```
/* Package Specification*/
CREATE OR REPLACE PACKAGE emp_admin
IS
  /*declaring the function */
  FUNCTION get_dept_name (p_deptno NUMBER DEFAULT 10)
       RETURN VARCHAR2;
  FUNCTION update_emp_sal (p_empno NUMBER, p_raise NUMBER)
       RETURN NUMBER;
 /* Declaring the Procedure */
  PROCEDURE hire_emp (
       p_empno     NUMBER,
       p_ename     VARCHAR2,
       p_job       VARCHAR2,
       p_sal       NUMBER,
       p_hiredate  DATE DEFAULT SYSDATE,
       p_comm      NUMBER DEFAULT 0,
       p_mgr       NUMBER,
       p_deptno    NUMBER DEFAULT 10
  );
  PROCEDURE fire_emp (p_empno NUMBER);
END emp_admin;
```

And Corresponding Package Body:

```
/* Package body for the 'emp_admin' package. */
CREATE OR REPLACE PACKAGE BODY emp_admin
IS
  /* Function that queries the 'dept' table based on the department Number and returns the
corresponding
     department name. */
  FUNCTION get_dept_name (p_deptno IN NUMBER DEFAULT 10)
    RETURN VARCHAR2
    IS
      v_dname   VARCHAR2 (14);
  BEGIN
    SELECT dname
    INTO v_dname
    FROM dept
    WHERE deptno = p_deptno;
    RETURN v_dname;
    EXCEPTION
    WHEN NO_DATA_FOUND
    THEN
      DBMS_OUTPUT.put_line ('Invalid department number ' || p_deptno);
      RETURN '';
  END get_dept_name;
```

```
  /* Function that updates an employee's salary based on the employee number and salary
increment/decrement
     passed as IN parameters.  Upon successful completion the function returns the new updated
salary.*/
  FUNCTION update_emp_sal (p_empno IN NUMBER, p_raise IN NUMBER)
        RETURN NUMBER
  IS
    v_sal   NUMBER := 0;
  BEGIN
    SELECT sal
    INTO v_sal
    FROM emp
    WHERE empno = p_empno;
    v_sal:= v_sal + p_raise;
    UPDATE emp
    SET sal = v_sal
    WHERE empno = p_empno;
    RETURN v_sal;
  EXCEPTION
    WHEN NO_DATA_FOUND
    THEN
      DBMS_OUTPUT.put_line ('Employee ' || p_empno || ' not found');
      RETURN -1;
    WHEN OTHERS
    THEN
      DBMS_OUTPUT.put_line ('The following is SQLERRM :');
      DBMS_OUTPUT.put_line (SQLERRM);
      DBMS_OUTPUT.put_line ('The following is SQLCODE :');
      DBMS_OUTPUT.put_line (SQLCODE);
      RETURN -1;
  END  update_emp_sal;
  /*Procedure that inserts a new employee record into the 'emp' table.*/
  PROCEDURE hire_emp (
        p_empno      NUMBER,
        p_ename      VARCHAR2,
        p_job        VARCHAR2,
        p_sal        NUMBER,
        p_hiredate   DATE DEFAULT SYSDATE,
        p_comm       NUMBER DEFAULT 0,
        p_mgr        NUMBER,
        p_deptno     NUMBER DEFAULT 10
      )
  AS
  BEGIN
     INSERT INTO emp (empno,  ename,   job,    sal,   hiredate,   comm,   mgr,   deptno)
            vALUES (p_empno, p_ename, p_job, p_sal, p_hiredate, p_comm, p_mgr, p_deptno );
  END hire_emp;
  /*Procedure that deletes an employee record from the 'emp' table based on the employee
number.*/
```

```
  PROCEDURE fire_emp (p_empno NUMBER)
  AS
  BEGIN
    DELETE FROM emp
    WHERE empno = p_empno;
    DBMS_OUTPUT.put_line ('Deleted successfully..');
  END fire_emp;
END;
```

Execution Process:- To execute the function or procedure with in the package we must specify "**<package_name>.<function_name> (or) <package_name>.<procedure_name>"** **For Function:-**

```
DECLARE
  v_result   VARCHAR2 (100);
BEGIN
 /*Here we must specify the function name along with its corresponding package name*/
  v_result:= emp_admin.get_dept_name (20);
  DBMS_OUTPUT.put_line ('Department Name =>' || v_result);
END;
```

Output:-

anonymous block completed /* Script Output */

Department Name =>RESEARCH /* DBMS output */

For Procedure:-

```
BEGIN
 /*Here we must specify the procedure  name along with its corresponding package name*/
  emp_admin.fire_emp (3567);
END;
```

Output:-

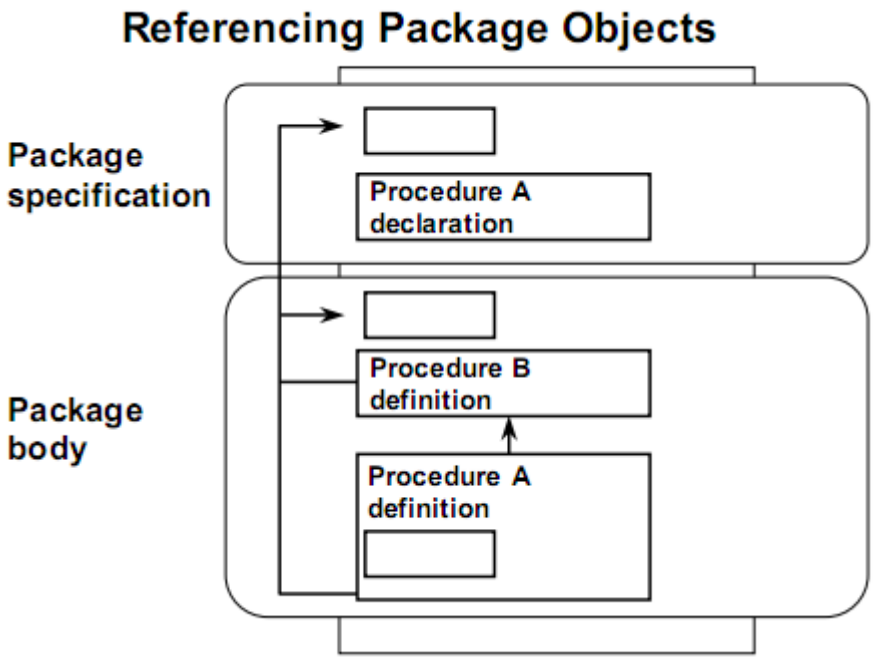
anonymous block completed /* Script Output */

Deleted successfully.. /* DBMS output */

- You create a package in two parts: first the package specification, and then the package body.
- Public package constructs are those that are declared in the package specification and defined in the package body.
- Private constructs are those that are defined solely with in the package body.

Scope of the construct	Description	Placement with in a package
Public	Can be referenced from any oracle server environment.	Declared within the package specification and may be defined within the package body.
Private	Can be referenced only by other constructs which are	Declared and defined within the package body.

	part of same package.	



The oracle server stores the specification and body of a package separately. This enables you to change the definition of a program construct in the package body without causing the oracle server to validate other schema objects that call or reference the program construct.

Local variables and global variables:-
Local variable:-
A variable defined with in a subprogram that is not visible to external users.
Private variable mean you can define variables in a package body. These variables can be accessed only by the other objects in the same package.
They are not visible to subprograms or objects outside of the package.

Global Variable:-
A variable or subprogram that can be referenced (and changed) outside the package and is visible to external users. Global package items must be declared in the package specification.

PL/SQL Tutorial - Advantages Of Packages

Packages provide an alternative to creating procedures and functions as standalone schema object, and they offer several benefits.

Modularity:-
You encapsulate logically related programming structures in a named module. Each package is easy to understand, and the interface between packages is simple, clear and well defined.

Easier Application Design:-

You can code and compile a specification without its body. Then stored subprograms that reference the package can compile as well.

You need not define the package body fully until you are ready to complete the application.

Hiding the Information:-

You can decide constructs are public(visible and accessible) or private(hidden and inaccessible).

Only the declarations In the package specification are visible and accessible to applications.

The package body hides the definition of the private constructs so that only the package affected if the definition changes.

Better Performance:-

When you call a packaged subprogram the first time ,the entire package is loaded into memory. this way, later calls to related subprograms in the package require no further disk I/O.

Packages subprograms also stop cascading dependencies and so avoid unnecessary compilation.

Overloading:-

With packages you can overload procedures and functions, which means you can create multiple subprograms with the same name in the same package, each taking parameters of different number of datatype.

PL/SQL Tutorial - What is a collection?

- A **collection** is a composite type used to store group/array of values of same data type. Collections are mainly used to simplify the code. Storing values in collection will allow us to loop through each element with ease, referencing each one by index.
- Collections are list, which may be ordered or unordered.
- Ordered list are indexed by unique subscripts.
- Unordered lists are indexed by unique identifiers, which may be numbers, hash values, or string names.
- Collections improve application performance because we cache data that is regularly used which will reduce the calls to database.

Types of Collections:

1. Varrays.
2. Nested tables.
3. Index – by tables (Associate arrays).

PL/SQL Tutorial - Collections Methods

A collection method is a built-in function or procedure that operates on collections and is called using dot notation.

Following are the functions used with collections.

- EXISTS
- COUNT
- LIMIT
- FIRST AND LAST
- PRIOR AND NEXT
- EXTEND
- TRIM
- DELETE

Syntax:-collection_name.method_name [(parameters)]
1

- Collection methods cannot be called from SQL statements.
- Only the EXISTS method can be used on a NULL collection.
- All other methods applied on a null collection raise the COLLECTION_IS_NULL error.

COUNT:

- COUNT method returns the number of elements with the space allocated in Varrays and Nested tables.
- The COUNT method displays number of elements in associate arrays.
- COUNT can be smaller than LIMIT for Varrays.

Example:-

```
DECLARE
/*Index by table type*/
  TYPE enametab IS TABLE OF emp%ROWTYPE
    INDEX BY BINARY_INTEGER;
  emptab    enametab;
/*Nested Table Type*/
  TYPE tab_type IS TABLE OF VARCHAR2 (10);
  tabtype   tab_type    := tab_type ();
/* Varray Type */
  TYPE varray_type IS VARRAY (3) OF VARCHAR2 (10);
 /* directly assign the values */
  v_type    varray_type := varray_type ('MILLER', 'FORD', 'TURNER');
  CURSOR c_emp
  IS
    SELECT *
      FROM emp;
  v_num     NUMBER;
BEGIN
  v_num := 1;
 /*allocate the spaces*/
  tabtype.EXTEND (3);
```

```
  tabtype (1) := 'SCOTT';
  tabtype (2) := 'KING';
  tabtype (3) := 'SMITH';
  tabtype.EXTEND (3);
  tabtype (4) := 'SCOTT';
  tabtype (5) := 'KING';
  tabtype (6) := 'SMITH';
  FOR emp_rec IN c_emp
  LOOP
    emptab (v_num) := emp_rec;
    v_num := v_num + 1;
  END LOOP;
 /*count the number of spaces allocated to nested table*/
  DBMS_OUTPUT.put_line (   'Total spaces allocated by Nested Table =>'
                                || tabtype.COUNT
                                );
/*count the number of spaces allocated to Varray */
  DBMS_OUTPUT.put_line ('Total spaces allocated by Varray=>' || v_type.COUNT);
/*count the number of elements In index by table*/
  DBMS_OUTPUT.put_line (   'Total Count of elements in Index by Table=>'
                                || emptab.COUNT
                                );
END;
```

Output:-

```
anonymous block completed
Total spaces allocated by Nested Table =>6
Total spaces allocated by Varray=>3
Total Count of elements in Index by Table=>9
```

EXISTS:

- EXISTS is a method determines whether or not an element is found in a collection. It takes a single formal parameter that is overloaded.
- The parameter data types are PLS_INTERGER, VARCHAR2, and LONG.
- If the collection is a null element structure, the EXISTS method will raise COLLECTION_IS_NULL exception.

Example:-

```
DECLARE
/*Nested Table Type*/
  TYPE tab_type IS TABLE OF VARCHAR2 (10);
/*declare the variable and initialize the variable*/
  tabtype   tab_type := tab_type ();
BEGIN
 /*allocate the spaces*/
  tabtype.EXTEND (3);
  tabtype (1) := 'SCOTT';
  tabtype (2) := 'KING';
  tabtype (3) := 'SMITH';
  tabtype.EXTEND (3);
  tabtype (4) := 'SCOTT';
  tabtype (5) := 'KING';
  tabtype (6) := 'SMITH';
```

```
/*checks the 1st space. If the 1st space having element then its return true otherwise false*/
  IF (tabtype.EXISTS (1))
  THEN
    DBMS_OUTPUT.put_line ('Element found for 1st space');
  END IF;
  IF (tabtype.EXISTS (7))
  THEN
    DBMS_OUTPUT.put_line ('Element found for 7th space');
  ELSE
    DBMS_OUTPUT.put_line ('Element not found for 7th space');
  END IF;
END;
```

Output:-

```
anonymous block completed
Element found for 1st space
Element not found for 7th space
```

LIMIT:

- The LIMIT method returns the highest allowed subscript value in Varray.

Example:-

```
DECLARE
/*declare the Varray */
  TYPE varray_type IS VARRAY (3) OF VARCHAR2 (10);
/*Initialize the Varray and assign the values directly to the varray type variable */
  v_type   varray_type := varray_type ('MILLER', 'FORD', 'TURNER');
BEGIN
/*Print the limit of varray*/
  DBMS_OUTPUT.put_line ('Limit of Varray is=>' || v_type.LIMIT);
END;
Output:-
anonymous block completed
Limit of Varray is=>3
```

FIRST and LAST:-

- The **FIRST** method will display returns the lowest subscript value in a collection.
- If it is a numeric index, it returns a PLS_INTEGER.If it’s an associative array, it returns a VARCHAR2 or LONG data type.
- You can’t use the FIRST method in a range for-loop when the index is non-numeric.
- The **LAST** method will display returns the highest subscript value in a collection.
- If it is a numeric index, it returns a PLS_INTEGER.If it’s an associative array, it returns a VARCHAR2 or LONG data type.
- You can’t use the LAST method in a range for-loop when the index is non-numeric.

Example:-

```
DECLARE
/*Index by table type*/
  TYPE enametab IS TABLE OF emp%ROWTYPE
    INDEX BY BINARY_INTEGER;
  emptab    enametab;
/*Nested Table Type*/
  TYPE tab_type IS TABLE OF VARCHAR2 (10);
```

```
  tabtype   tab_type    := tab_type ();
/* Varray Type */
  TYPE varray_type IS VARRAY (3) OF VARCHAR2 (10);
  /* directly assign the values */
  v_type    varray_type := varray_type ('MILLER', 'FORD', 'TURNER');
  CURSOR c_emp
  IS
    SELECT *
      FROM emp;
  v_num     NUMBER;
BEGIN
  v_num := 1;
  /*allocate the spaces*/
  tabtype.EXTEND (3);
  tabtype (1) := 'SCOTT';
  tabtype (2) := 'KING';
  tabtype (3) := 'SMITH';
  tabtype.EXTEND (3);
  tabtype (4) := 'SCOTT';
  tabtype (5) := 'KING';
  tabtype (6) := 'SMITH';
  FOR emp_rec IN c_emp
  LOOP
    emptab (v_num) := emp_rec;
    v_num := v_num + 1;
  END LOOP;
  /*get the lowest subscripted value and high subscripted value in Nested table*/
  DBMS_OUTPUT.put_line ('---Nested Table---');
  DBMS_OUTPUT.put_line (   'the first value ('|| tabtype.FIRST|| ') =>'|| tabtype (tabtype.FIRST)
                                 );
  DBMS_OUTPUT.put_line (   'the last value ('|| tabtype.LAST|| ') =>'|| tabtype (tabtype.LAST)
                                 );
/* get the lowest subscripted value and high subscripted value in Varray */
  DBMS_OUTPUT.put_line ('---Varrays---');
  DBMS_OUTPUT.put_line (   'the first value ('|| v_type.FIRST|| ') =>'|| v_type (v_type.FIRST)
                                 );
  DBMS_OUTPUT.put_line (   'the last value ('|| v_type.LAST|| ') =>'|| v_type (v_type.LAST)
                                 );
/* get the lowest subscripted value and high subscripted value in Index by table */
  DBMS_OUTPUT.put_line ('---Index By Table---');
  DBMS_OUTPUT.put_line (   'the first value of empno('|| emptab.FIRST|| ')=>'|| emptab
(emptab.FIRST).empno
                                  );
  DBMS_OUTPUT.put_line (   'the last value of empno('|| emptab.LAST|| ')=>'|| emptab
(emptab.LAST).empno
                    );
END;
```

Output:-

```
anonymous block completed
---Nested Table---
the first value (1) =>SCOTT
```

```
the last value (6) =>SMITH
---Varrays---
the first value (1) =>MILLER
the last value (3) =>TURNER
---Index By Table---
the first value of empno(1)=>7839
the last value of empno(9)=>7934
```

NEXT and PRIOR:-

- The **NEXT** method uses the subscript to find the next higher subscript in the collection.
- If there is no higher subscript value, NEXT return NULL.
- The **PRIOR** method uses the subscript to find the next lower subscript in the collection.
- If there is no higher subscript value, PRIOR return NULL.

Example:-

```
DECLARE
/*Index by table type*/
  TYPE enametab IS TABLE OF emp%ROWTYPE
    INDEX BY BINARY_INTEGER;
  emptab    enametab;
/*Nested Table Type*/
  TYPE tab_type IS TABLE OF VARCHAR2 (10);
  tabtype   tab_type    := tab_type ();
/* Varray Type */
  TYPE varray_type IS VARRAY (3) OF VARCHAR2 (10);
  /* directly assign the values */
  v_type    varray_type := varray_type ('MILLER', 'FORD', 'TURNER');
  CURSOR c_emp
  IS
    SELECT *
      FROM emp;
  v_num     NUMBER;
BEGIN
  v_num:= 1;
  /*allocate the spaces*/
  tabtype.EXTEND (3);
  tabtype (1) := 'SCOTT';
  tabtype (2) := 'KING';
  tabtype (3) := 'SMITH';
  tabtype.EXTEND (3);
  tabtype (4) := 'SCOTT';
  tabtype (5) := 'TURNER';
  tabtype (6) := 'SMITH';
  FOR emp_rec IN c_emp
  LOOP
    emptab (v_num) := emp_rec;
    v_num:= v_num + 1;
  END LOOP;
  /*we try to find out the next higher subscript value and lower subscript value in Nested Table*/
  DBMS_OUTPUT.put_line ('---Nested Table---');
```

```
  DBMS_OUTPUT.put_line ('the Next value ('|| tabtype.NEXT (tabtype.FIRST)|| ') =>'|| tabtype
(tabtype.NEXT (tabtype.FIRST)));
  DBMS_OUTPUT.put_line('the previous value('||tabtype.PRIOR(tabtype.LAST)||')
                                                =>'||tabtype (tabtype.PRIOR (tabtype.LAST))
                              );
/*we try to find out the next higher subscript value and lower subscript value in Varrays*/
  DBMS_OUTPUT.put_line ('---Varrays---');
  DBMS_OUTPUT.put_line ('the Next value ('|| v_type.NEXT (v_type.FIRST)|| ') =>'||
                                                v_type (v_type.NEXT (v_type.FIRST))
                               );
  DBMS_OUTPUT.put_line ('the previous value ('|| v_type.PRIOR (v_type.LAST)|| ') =>'||
                                                  v_type (v_type.PRIOR (v_type.LAST))
                               );
/*we try to find out the next higher subscript value and lower subscript value in Index By table*/
  DBMS_OUTPUT.put_line ('---Index By Table---');
  DBMS_OUTPUT.put_line ('the next value of empno('|| emptab.NEXT(emptab.FIRST)|| ')=>'||
                                                    emptab (emptab.NEXT
(emptab.FIRST)).empno
                              );
  DBMS_OUTPUT.put_line ('the previous value of empno('|| emptab.PRIOR(emptab.LAST)|| ')=>'||
                                                      emptab (emptab.PRIOR
(emptab.LAST)).empno
                               );
END;
```

Output:-

```
anonymous block completed
---Nested Table---
the Next value (2) =>KING
the previous value(5)
                                                =>TURNER
---Varrays---
the Next value (2) =>FORD
the previous value (2) =>FORD
---Index By Table---
the next value of empno(2)=>7698
the previous value of empno(8)=>7900
```

EXTEND:-

- The **EXTEND** allocates the space for new element in a collection. It is used to allocate space before adding a value to the collection.
- EXTEND will fail it attempts to exceed the LIMIT of a varray.
- And it takes a single formal parameter; it is optional (parameter).

Example:-

```
DECLARE
/*Nested Table Type*/
  TYPE tab_type IS TABLE OF VARCHAR2 (10);
  tabtype   tab_type    := tab_type ();
/* Varray Type */
  TYPE varray_type IS VARRAY (3) OF VARCHAR2 (10);
 /* directly assign the values */
  v_type    varray_type:= varray_type ('MILLER', 'FORD', 'TURNER');
```

```
BEGIN
  /*allocate the spaces for each element individually*/
   tabtype.EXTEND;
   tabtype (1) := 'SCOTT';
   tabtype.EXTEND;
   tabtype (2) := 'KING';
   tabtype.EXTEND;
   tabtype (3) := 'SMITH';
/*we try to allocate the spaces for 3 elements at a time */
   tabtype.EXTEND (3);
   tabtype (4) := 'SCOTT';
   tabtype (5) := 'KING';
   tabtype (6) := 'SMITH';
 /*count the number of spaces allocated to nested table*/
   DBMS_OUTPUT.put_line (   'Total spaces allocated by Nested Table =>'
                                    || tabtype.COUNT
                                    );
/*count the number of spaces allocated to Varray */
   DBMS_OUTPUT.put_line ('Total spaces allocated by Varray=>' || v_type.COUNT);
END;
```

Output:-

anonymous block completed

Total spaces allocated by Nested Table =>6

Total spaces allocated by Varray=>3

DELETE:-

- The **DELETE** will remove the element with subscript.
- The other version (DELETE (n, m)) will delete a continuous inclusive range of elements from a collection.

Example:-

```
DECLARE
/*Nested Table Type*/
   TYPE tab_type IS TABLE OF VARCHAR2 (10);
   tabtype   tab_type    := tab_type ();
/*  Varray Type */
   TYPE varray_type IS VARRAY (3) OF VARCHAR2 (10);
 /*  directly assign the values */
   v_type    varray_type := varray_type ('MILLER', 'FORD', 'TURNER');
BEGIN
  /*allocate the spaces*/
   tabtype.EXTEND;
   tabtype (1) := 'SCOTT';
   tabtype.EXTEND;
   tabtype (2) := 'KING';
   tabtype.EXTEND;
   tabtype (3) := 'SMITH';
   tabtype.EXTEND (3);
   tabtype (4) := 'SCOTT';
   tabtype (5) := 'KING';
   tabtype (6) := 'SMITH';
 /*count the number of spaces allocated to nested table*/
```

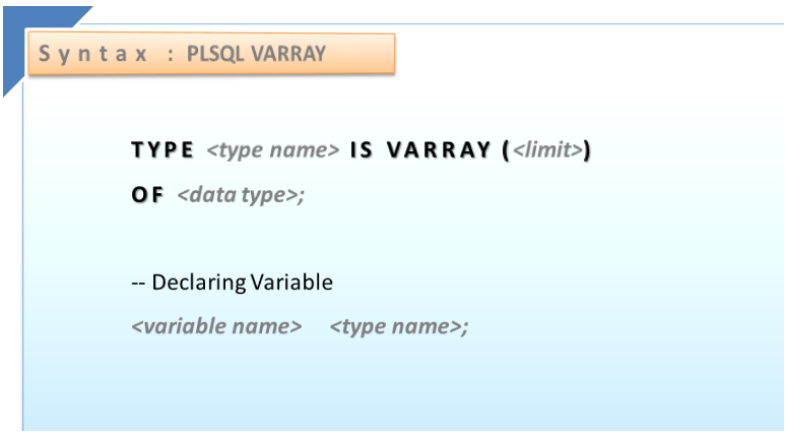
```
  DBMS_OUTPUT.put_line (   'Total spaces allocated by Nested Table before delete =>' ||
tabtype.count );
   tabtype.DELETE;
  DBMS_OUTPUT.put_line (   'Total spaces allocated by Nested Table After delete =>'||
tabtype.count
                                    );
 /*count the number of spaces allocated to Varray */
  DBMS_OUTPUT.put_line ('Total spaces allocated by Varray before delete =>' || v_type.count);
   v_type.DELETE;
  DBMS_OUTPUT.put_line ('Total spaces allocated by Varray After delete=>' || v_type.count);
END;
```

Output:-

anonymous block completed
Total spaces allocated by Nested Table before delete =>6
Total spaces allocated by Nested Table After delete =>0
Total spaces allocated by Varray before delete =>3
Total spaces allocated by Varray After delete=>0

PL/SQL Tutorial - VARRAYS

- **Varrays** stands for variable-size array.
- A Varrays has a fixed limit on its size, specified as part of the declaration. Elements are inserted into varray starting at index 1, up to maximum length declared in the varray type.
- Data in varray can be either manipulated as a whole or individually as elements.
- Varrays cannot have any gaps in the index sequence. Index always starts from 1 not 0;
- We would use a varray whenever the physical size of the collection is static. These are like the traditional arrays in other programming languages.
- We can use Varrays in SQL as well as PL/SQL.



PL/SQL Example1:-

```
DECLARE
      /* define a varray with 5 rows */
    TYPE char_type IS VARRAY (5) OF VARCHAR2 (10);
      /*calls the Varray constructor */
    c_list  char_type := char_type ();
BEGIN   /* allocates space in the varray */
   c_list.EXTEND;
   c_list (1) := 'SCOTT';
```

```
            /* Can have up to 10 characters */
  c_list.EXTEND;
  c_list (2) := 'TIGER';
          /* print the varray elements by using its methods FIRST and LAST */
  FOR i IN c_list.FIRST.. c_list.LAST
  LOOP
  DBMS_OUTPUT.put_line (c_list (i));
  END LOOP;
END;
```

Output:-

anonymous block completed /* Script Output */

SCOTT /*DBMS Output *?

TIGER

Example2:-

```
DECLARE
        /*define a varray of "emp" type*/
   TYPE myarray IS VARRAY (20) OF emp%ROWTYPE;  /*declare and initialize a null set of rows*/
   v_list    myarray := myarray ();
   CURSOR cu_emp  IS  SELECT *  FROM emp;
   v_index   NUMBER;
BEGIN    v_index := 1;
   FOR i IN cu_emp    LOOP   /*initialize row*/
   v_list.EXTEND;
   SELECT * INTO v_list (v_index) FROM emp
   WHERE empno = i.empno;    /*print the contents*/
 DBMS_OUTPUT.put_line ('empno = '|| v_list (v_index).empno|| ' ename ='|| v_list
(v_index).ename);
        /*increment the index value*/
   v_index := v_index+ 1;
   END LOOP;
END;
```

Output:-

anonymous block completed /Script Output */

```
/* DBMS OUtput is */
empno = 7839 ename =KING
empno = 7698 ename =BLAKE
empno = 7782 ename =CLARK
empno = 7499 ename =ALLEN
empno = 7521 ename =WARD
empno = 7654 ename =MARTIN
empno = 7844 ename =TURNER
empno = 7900 ename =JAMES
empno = 7934 ename =MILLER
/* the output might be different depending upon the availability of rows and updates */
```

Common Exceptions in Varrays:-

Example3:-

```
DECLARE
    /* define a varray with 5 rows */
 TYPE char_type IS VARRAY (5) OF VARCHAR2 (10);
```

```
    /*we declare a varray but not initialize so system will raise an error*/
 c_list   char_type;
BEGIN
   /* allocates space in the varray */
  c_list.EXTEND;
  c_list (1):= 'SCOTT';
            /* Can have up to 10 characters */
  c_list.EXTEND;
  c_list (2) := 'TIGER';
            /* print the varray elements by using its methods FIRST and LAST */
  FOR i IN c_list.FIRST.. c_list.LAST
  LOOP
    DBMS_OUTPUT.put_line (c_list (i));
  END LOOP;
END;
DECLARE
*
ERROR Report:
ORA-06531: Reference to uninitialized collection
ORA-06512: at line 8
```

This exception raises when you forgot to initialize the varray. So once after declaration of the varray variable and then initialize the variable is must otherwise system will rise above exception.

Example4:-

```
DECLARE
      /* define a varray with 5 rows */
  TYPE char_type IS VARRAY (3) OF VARCHAR2 (10);
  c_list   char_type := char_type ();
BEGIN
  c_list.EXTEND;
  c_list (1):= 'SCOTT';
      /* Can have up to 10 characters */
  c_list.EXTEND;
  c_list (2):= 'TIGER';
      /* before allocates space in the varray we try to use that variable then system will raise an error */
  c_list (3):= 'KING';
     /* print the varray elements by using its methods FIRST and LAST */
  FOR i IN c_list.FIRST.. c_list.LAST
  LOOP
    DBMS_OUTPUT.put_line (c_list (i));
  END LOOP;
END;
```

Output:-

```
Error report:
ORA-06533: Subscript beyond count
ORA-06512: at line 12
```

The exception means that subscript 3 is unavailable. It does not exist. While you defined the varray as three elements in size, you allocate the space for only two elements. Therefore the variable has only two valid subscripts, one and two.

Example5:-

```
DECLARE
    /* define a varray with 5 rows */
  TYPE char_type IS VARRAY (3) OF VARCHAR2 (10);
  c_list   char_type := char_type ();
BEGIN
    /* allocates space in the varray */
  c_list.EXTEND;
  c_list (1):= 'SCOTT';
    /* Can have up to 10 characters */
  c_list.EXTEND;
   c_list (2) := 'TIGER';
  c_list.EXTEND;
   c_list (3) := 'KING';
  c_list.EXTEND;
    /* print the varray elements by using its methods FIRST and LAST */
  FOR i IN c_list.FIRST.. c_list.LAST
  LOOP
    DBMS_OUTPUT.put_line (c_list (i));
  END LOOP;
END;
```

Output:-

Error report:

ORA-06532: Subscript outside of limit

ORA-06512: at line 14

In the above program we declare the varray size is 3 then we try to allocate the space for the 4th element then system will raise an error.

PL/SQL Tutorial - Nested Tables

- A **Nested table** is like a database table which has no limit on its size. Elements are inserted into nested table starting with index 1. Nested table can be created in both SQL and PLSQL.
- Nested table is nothing but a table with in a table.
- A table is represented as a column with in another table.
- There is no limit to the number of rows in the nested table for each row in the main table.
- Basically it is used for mapping master-detail relationship between tables.
- We would use Nested tables when physical size of the collection is unknown and wanted to extend it dynamically.
- We can use Nested Tables in SQL as well as PL/SQL.
- Extend is a method that adds a new empty element to a nested table, this must be called first.
- We cannot delete individual elements from a Varray. But we can delete elements from a nested table using the built-in procedure DELETE. That might leave gaps in the index, but the built-in function NEXT lets you iterate over any series of subscripts.

Syntax : NESTED TABLE

```
TYPE <type name> IS TABLE OF <data type>;

-- Declaring Variable
<variable name>    <type name>;
```

PL/SQL Example1:-

```
DECLARE
      /* declare the table type */
  TYPE students_table IS TABLE OF VARCHAR2 (10);
      /* calls the nested table constructor */
  students   students_table := students_table ();
BEGIN
  students.EXTEND;
 /* can have up to 10 characters as stated above*/
  students (1):= 'SCOTT';
  students.EXTEND;
  students (2):= 'LUCAS';
  students.EXTEND;
  students (3):= 'SMITH';
            /* print the table type elements by using FIRST and LAST */
  FOR i IN students.FIRST .. students.LAST
  LOOP
    DBMS_OUTPUT.put_line (students (i));
  END LOOP;
END;
```

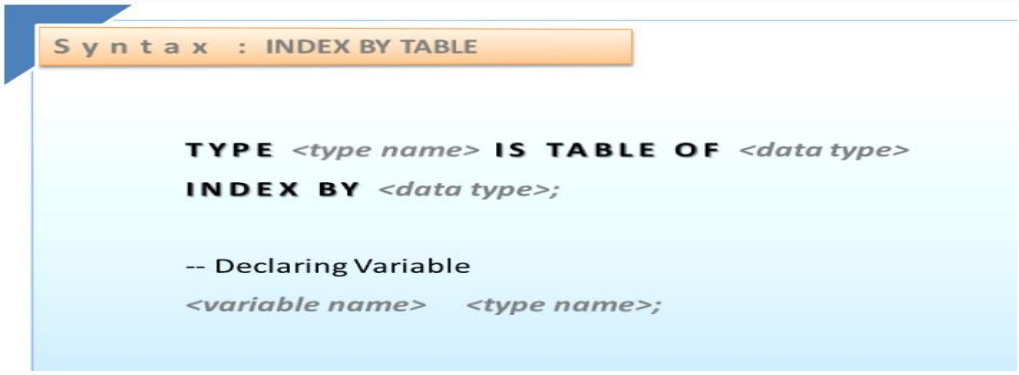
Output:-

```
anonymous block completed
SCOTT
LUCAS
SMITH
```

PL/SQL Tutorial - Index By Tables or Associate Arrays

- An **index-by table** has no limit on its size. Elements are inserted into index-by table whose index may start non-sequentially including negative integers.
- Associate Arrays can only be accessed in PL/SQL. (not in SQL)
- They are modeled as similar to database tables but they are not permanent tables i.e. they can be created and manipulated only in PL/SQL block.

- Index can be of any data type.
- We can store unlimited values.
- Suitable for storing and displaying the values of one column of a table given by a cursor.
- These Associate arrays do not require initialization and have no constructor syntax.
- No need to allocate space before assigning values (i.e. no extend method is required) as in Varrays and Nested tables.
- Associate arrays cannot be used in tables. They may be used only as programming structures i.e. in pl/sql.



Example1:-

```
DECLARE
    /* declaring the Index-by-table or associate array based on the "emp" table type*/
  TYPE enametab IS TABLE OF emp.ename%TYPE
  INDEX BY BINARY_INTEGER;
     /*assigning the tabletype variable into anther variable which same as table type variable*/
  enames   enametab;
    /*declaring the cursor*/
  CURSOR emp_cur
  IS
  SELECT ename FROM emp;
  v_num NUMBER;
BEGIN
   v_num:=1;
   FOR v_ec IN emp_cur
   LOOP
      /*assign the index based on the ename's of the "emp" table*/
   enames (v_num):= v_ec.ename;
   v_num:=v_num+1;
   END LOOP;
      /*print the elements in "enames" type by using table type methods i.e. FIRST and LAST*/
   FOR i IN enames.FIRST.. enames.LAST
   LOOP
     DBMS_OUTPUT.put_line (enames (i));
   END LOOP;
END;
```

Output:-

```
anonymous block completed
KING    /*DBMS output */
BLAKE
CLARK
```

ALLEN
WARD
MARTIN
TURNER
JAMES
MILLER

PL/SQL Tutorial - Differences Among collections

- Varrays has limit, nested tables and index-by tables has no limit.
- Varrays and nested tables must be initialized before assignment of elements, in index-by tables we can directly assign elements.
- Varrays and nested tables stored in database, but index-by tables can not.
- Keys must be positive in case of nested tables and varrays, in case of index-by tables keys can be positive or negative.
- Referencing nonexistent elements raises SUBSCRIPT_BEYOND_COUNT in both nested tables and varrays, but in case of index-by tables NO_DATA_FOUND raises.
- Keys are sequential in both nested tables and varrays, non-sequential in index-by tables.
- Individual indexes can be deleted in both nested tables and index-by tables, but cannot be deleted in varrays.
- Nested table stores unlimited data because it stored outside the table, but varray stored limited data because it stored with in the table.
- Nested table allows DML but varray do not allow DML.
- Index by table and nested tables are un-bounded but varray are fixed length.
- Collections will not support constraints.
- LONG, RAW, LOB datatype are not valid in collections.
- Collections will not support developer6i,VB(GUI tools).
- JAVA supports collections.

PL/SQL Tutorial - What is a Trigger?

Definition: – A set of PL/SQL statements stored permanently in database and automatically activated when ever an event raising statement (DML) is executed.

- They are stored in USER_TRIGGERS system table
- They are used to impose business rules or user defined restrictions on table columns.
- They are also activated when tables are manipulated by other application software tools.
- They provide high security.
- It will execute implicitly whenever the triggering event happens and trigger does not accept arguments.
- The act of executing a trigger is known as *firing*
- The trigger event can be a DML (INSERT, UPDATE, or DELETE) operation on database table or certain kinds of views; or system event, such as database startup or shutdown, and certain kinds of DDL operations.
- INSTEAD OF is used to create a trigger on a view. before and after cannot be used to create a trigger on a view

Trigger Parts: – Four parts are there.

1. Triggering event
2. Trigger types
3. Trigger restriction
4. Trigger body

1. **Triggering event**:- Indicates when to activate the trigger

BEFORE –INSERT/UPDATE/DELETE

AFTER –INSERT/UPDATE/DELETE

2. **Trigger types**:- Two types of Triggers are there
   1. Row level trigger- Activates the trigger for every row manipulated by DML statement
   2. Statement level trigger- Activates the trigger only one's for one DML statement(default type).

3. **Trigger Restriction**: – Used to stop the activation of trigger based on condition. If condition is TRUE trigger is active.

4. **Trigger body**: – A set of PL/SQL statements.

Syntax :TRIGGER

```
CREATE [OR REPLACE] TRIGGER <trigger name>
BEFORE / AFTER
INSERT / UPDATE / DELETE
[ OF <column> ] ON <table name>
[ REFERENCING :OLD AS o :NEW AS n ]
[FOR EACH ROW WHEN <condition> ]
DECLARE
    <declaration section>
BEGIN
    <execution section>
EXCEPTION
    <exception section>
END ;
```

NEW and OLD (pseudo columns):-

- Used to retrieve data from DML statement temporary buffer to trigger body
- Valid with ROW level trigger only
- NEW supports with

Explanation About syntax:-

1. CREATE [OR REPLACE] TRIGGER <trigger_name> – This clause creates a trigger with the given name or overwrites an existing trigger with the same name.

2. {BEFORE | AFTER | INSTEAD OF } – This clause indicates at what time the trigger should get fired. i.e. for example: before or after updating a table. INSTEAD OF is used to create a trigger on a view. Before and after cannot be used to create a trigger on a view.

3. {INSERT [OR] | UPDATE [OR] | DELETE} – This clause determines the triggering event. More than one triggering events can be used together separated by OR keyword. The trigger gets fired at all the specified triggering event.

4. [OF column_name] – This clause is used with update triggers. This clause is used when you want to trigger an event only when a specific column is updated.

5. CREATE [OR REPLACE] TRIGGER <trigger_name> – This clause creates a trigger with the given name or overwrites an existing trigger with the same name.

6. [ON table_name] – This clause identifies the name of the table or view to which the trigger is associated.

7. [REFERENCING OLD AS o NEW AS n] – This clause is used to reference the old and new values of the data being changed. By default, you reference the values as: old.column_name or: new.column_name. The reference names can also be changed from old (or new) to any other user-defined name. You cannot reference old values when inserting a record, or new values when deleting a record, because they do not exist.

8. [FOR EACH ROW] – This clause is used to determine whether a trigger must fire when each row gets affected (i.e. a Row Level Trigger) or just once when the entire sql statement is executed(i.e. statement level Trigger).

9. WHEN (condition) – This clause is valid only for row level triggers. The trigger is fired only for rows that satisfy the condition specified.

PL/SQL Tutorial - ROW LEVEL TRIGGER

- The trigger body executes once for each row affected by the triggering event. A row level trigger is not executed if the triggering event affects no rows.
- Row level triggers are useful if the trigger action depends on data of rows that are affected or on data provided by the triggering event itself.

Example:-1

/* here we are going to write trigger for automatically updating the 'ename' when ever we insert the record into 'emp' which is shown below*/

```
SQL>CREATE OR REPLACE TRIGGER up_ename
  BEFORE INSERT
  ON emp
  FOR EACH ROW
BEGIN
  :NEW.ename := UPPER (:NEW.ename);
END;
– Trigger created.
```

Now we are going to insert record into 'emp' table

/* here we are going to insert 'ename' in LOWER case*/

```
SQL> INSERT INTO emp (empno, ename,  sal,  comm, deptno)
            VALUES (7711, 'andrew',3500, 500,  10);
Output:-
SELECT * FROM emp WHERE empno=7711;
7711     ANDREW 3500  500       10
```

/* here we can see that ename is in UPPER case, now we can conclude that above trigger what we had written is fired at this event*/

Example2:-

/*here we trying to decrease the salary by updating then below trigger will raise*/

```
CREATE OR REPLACE TRIGGER up_sal BEFORE UPDATE ON emp FOR EACH ROW
BEGIN
   IF :NEW.sal<:OLD.sal THEN
      RAISE_APPLICATION_ERROR(-20000,'updating salary must greater than existing salary');
   END IF;
END;
```

Now we are going to decrease the 'sal'

```
UPDATE emp SET sal=sal-1000 WHERE empno=7839;
```

Output:-

/* here we can see that trigger raised*/

Error report:
SQL Error: ORA-20000: updating salary must greater than existing salary
ORA-06512: at "APPS.UP_SAL"
ORA-04088: error during execution of trigger 'APPS.UP_SAL'

PL/SQL Tutorial - STATEMENT LEVEL TRIGGER

Statement level Trigger:-

Activates the trigger for only once for one DML statement.

A statement trigger is fired once on behalf of the triggering event, even if no rows are affected at all.

Statement triggers are useful if the trigger action does not depend on the data from rows that are affected or on data provided by the triggering event itself: for example, a trigger that performs a complex security check on the current user.

Example:-

/*here we are trying to INSERT record INTO 'emp' table in week ends…. Below trigger will fire when we try to insert record in week ends*/

```
CREATE OR REPLACE TRIGGER sec_emp
 BEFORE INSERT ON emp
BEGIN
      IF TO_CHAR (SYSDATE,'DY') in ('SAT','SUN') THEN
          RAISE_APPLICATION_ERROR (-20101,'in week ends we can not insert record');
      END IF;
END;
```

Now are trying to insert into emp in week ends and we can see the output below

/* Before inserting change the system date to saturday or sunday date */ SQL>INSERT INTO EMP(EMPNO, ENAME, DEPTNO) VALUES (7913, 'mike', 10); Output:- Error report: SQL Error: ORA-20101: in week ends we can not insert record ORA-06512: at "APPS.SEC_EMP" ORA-04088: error during execution of trigger 'APPS.SEC_EMP'

RAISE_APPLICATION_ERROR:- it is built in function that stops the DML operation and displays the error message.

PL/SQL Tutorial - Trigger Execution Hierarchy

PL/SQL Trigger Execution Hierarchy
The following is the sequence in which different triggers are fired.

- **Before statement level**
- **Before row level**
- **After row level**
- **After statement level**

For Example: Let's create a table 'test' to store messages when triggers are fired.

```
CREATE TABLE test
(Message varchar2(2000));
```

1) BEFORE UPDATE, Statement Level: This trigger will insert a record into the table 'test' before a sql update statement is executed, at the statement level.

```
CREATE OR REPLACE TRIGGER emp_sal_upd_stmt_level_trigBEFORE UPDATE ON emp
BEGIN
  DBMS_OUTPUT.PUT_LINE('Updating employees ');
  INSERT INTO test values('Stmt level Before update - Updating employees ');
END;
```

2) BEFORE UPDATE, Row Level: This trigger will insert a record into the table 'test' before each row is updated.

```
CREATE OR REPLACE TRIGGER emp_sal_upd_stmt_level_trig
BEFORE UPDATE ON emp
FOR EACH ROW
DECLARE
   sal_diff      NUMBER;
BEGIN
   sal_diff := :NEW.sal - :OLD.sal;
   DBMS_OUTPUT.PUT_LINE('Updating employee ' || :OLD.empno);
   DBMS_OUTPUT.PUT_LINE('..Old salary: ' || :OLD.sal);
   DBMS_OUTPUT.PUT_LINE('..New salary: ' || :NEW.sal);
   DBMS_OUTPUT.PUT_LINE('..Raise     : ' || sal_diff);
Insert into test values('Row level Before update - Updating employee ' || :OLD.empno);
END;
/
```

3) AFTER UPDATE, Statement Level: This trigger will insert a record into the table 'product_check' after a sql update statement is executed, at the statement level.

```
 CREATE or REPLACE TRIGGER emp_sal_aftr_upd_stmt_level_trig
 AFTER
 UPDATE ON emp
 BEGIN
 DBMS_OUTPUT.PUT_LINE('Updating employees ');
 INSERT INTO test values('Stmt level After update - Updating employees ');
 END;
/
```

4) AFTER UPDATE, Row Level: This trigger will insert a record into the table 'product_check' after each row is updated.

```
CREATE OR REPLACE TRIGGER emp_sal_aftr_upd_row_level_trig
AFTER UPDATE ON emp
FOR EACH ROW
DECLARE
   sal_diff      NUMBER;
BEGIN
   sal_diff := :NEW.sal - :OLD.sal;
   DBMS_OUTPUT.PUT_LINE('Updating employee ' || :OLD.empno);
   DBMS_OUTPUT.PUT_LINE('..Old salary: ' || :OLD.sal);
   DBMS_OUTPUT.PUT_LINE('..New salary: ' || :NEW.sal);
   DBMS_OUTPUT.PUT_LINE('..Raise     : ' || sal_diff);
INSERT INTO test values('Row level after update - Updating employee ' || :OLD.empno);
END;
/
```

Now lets execute a update statement on table emp.

```
UPDATE emp SET sal = 2000
WHERE deptno = 10;
```

Let's check the data in test table to see the order in which the trigger is fired.

```
SQL>SELECT * FROM test;
```

PL/SQL Tutorial - Mutating Triggers

When we perform two DML operations at one's then it leads to 'Trigger Mutating' error while working with software tools.

If trigger applied on table1 (before insert on table1) performing any other DML or DRL on same table(table1) in trigger body causes "Mutating Error".

Example1:-

/*here we are deleting the row and at the same time we are selecting(DRL) the record then it leads to mutating error*/

```
CREATE OR REPLACE TRIGGER mut_trig
 AFTER DELETE ON emp FOR EACH ROW
 DECLARE
        x number;
 BEGIN
       SELECT sal
            INTO x
           FROM emp
        WHERE ename='SMITH';
 INSERT INTO emp(empno,ename,deptno) VALUES(9999,'APPS',10);
 DBMS_OUTPUT.PUT_LINE('SALARY =>'||x);
 END;
```

Now we can delete the record from 'emp'

```
SQL>DELETE FROM emp;
```

Output:-

Error starting at line 'x' in command:

DELETE FROM emp
Error report:
SQL Error: ORA-04091: table APPS.EMP is mutating, trigger/function may not see it
ORA-06512: at "APPS.MUT_TRIG", line 'y'
ORA-04088: error during execution of trigger 'APPS.MUT_TRIG'

Example2:-

/*here we are deleting the row and at the same time we are updating(DML) the record then it leads to mutating error*/

```
CREATE OR REPLACE TRIGGER mutuating_trig
 AFTER DELETE ON emp FOR EACH ROW
 DECLARE
 BEGIN
 UPDATE emp SET sal=sal+sal*.05
        WHERE ename='SMITH';
 END;
```

Now we can delete the record from ‘emp’

SQL>DELETE FROM emp;
Output:-
Error starting at line 'x' in command:
DELETE FROM emp
Error report:
SQL Error: ORA-04091: table APPS.EMP is mutating, trigger/function may not see it
ORA-06512: at "APPS.MUTUATING_TRIG", line 'y'
ORA-04088: error during execution of trigger 'APPS.MUTUATING_TRIG'